

Computer Architecture

CS 622 / EE 520

Spring 2026

Lecture 12
Shahid Masud

- Quick Review -> Scheduling Examples of Tomasulo and Scoreboard techniques
- (from Chap 3)
- 3.6 Reorder Buffer (ROB) and Hardware Based Speculation
- 3.7 Exploiting ILP Using Multiple Issue and Static Scheduling

Quiz 3
Next Week

Reading Notes

2019 EE 488

Dynamic Scheduling - Reorder Buffer (ROB) -

Myoungsoo Jung
Computer Division

Readings:
Chapter 3 of course textbook

Tomasulo Example

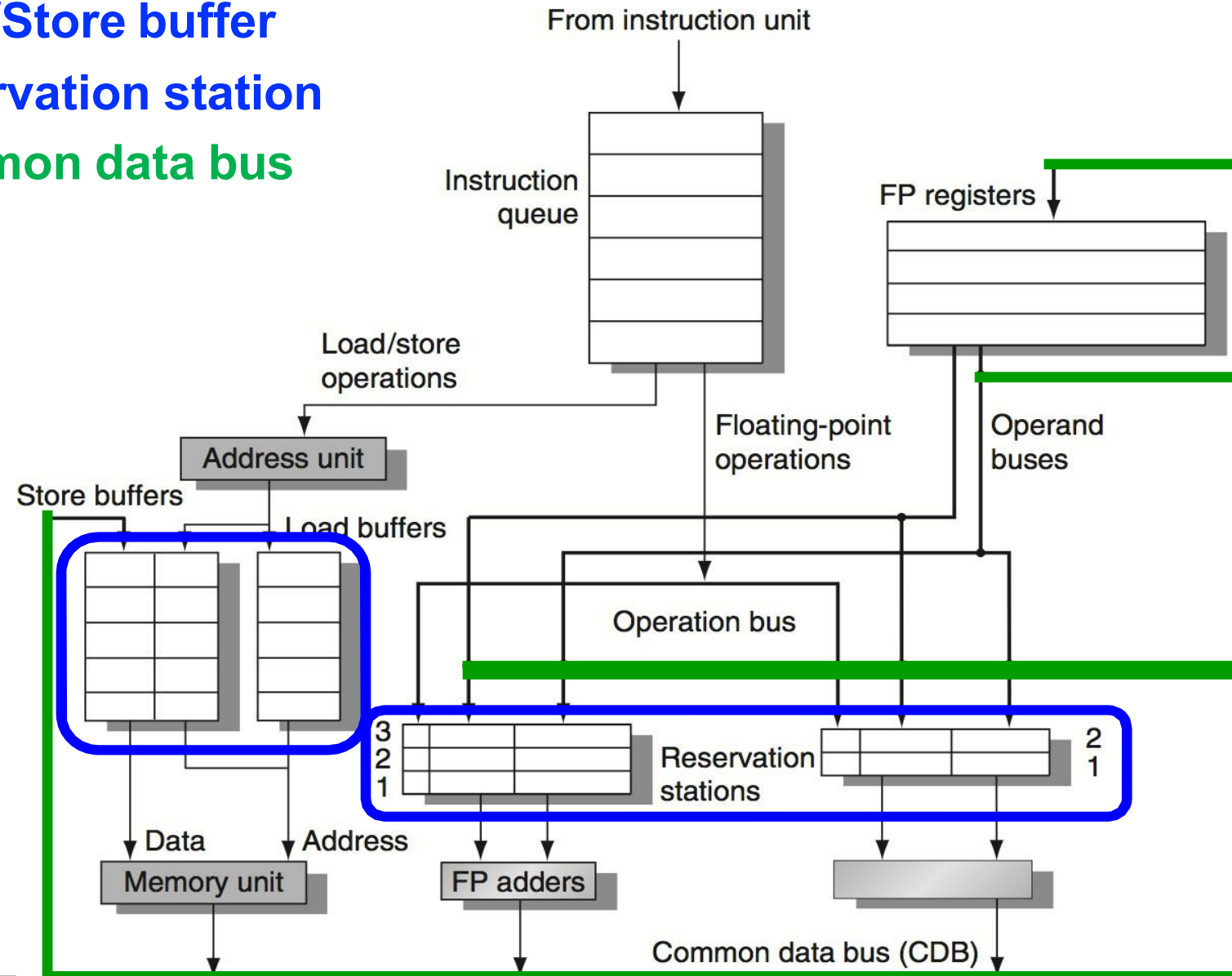
Reference:

<https://www.youtube.com/watch?v=EfwCfhXxNnI&list=WL&index=6>

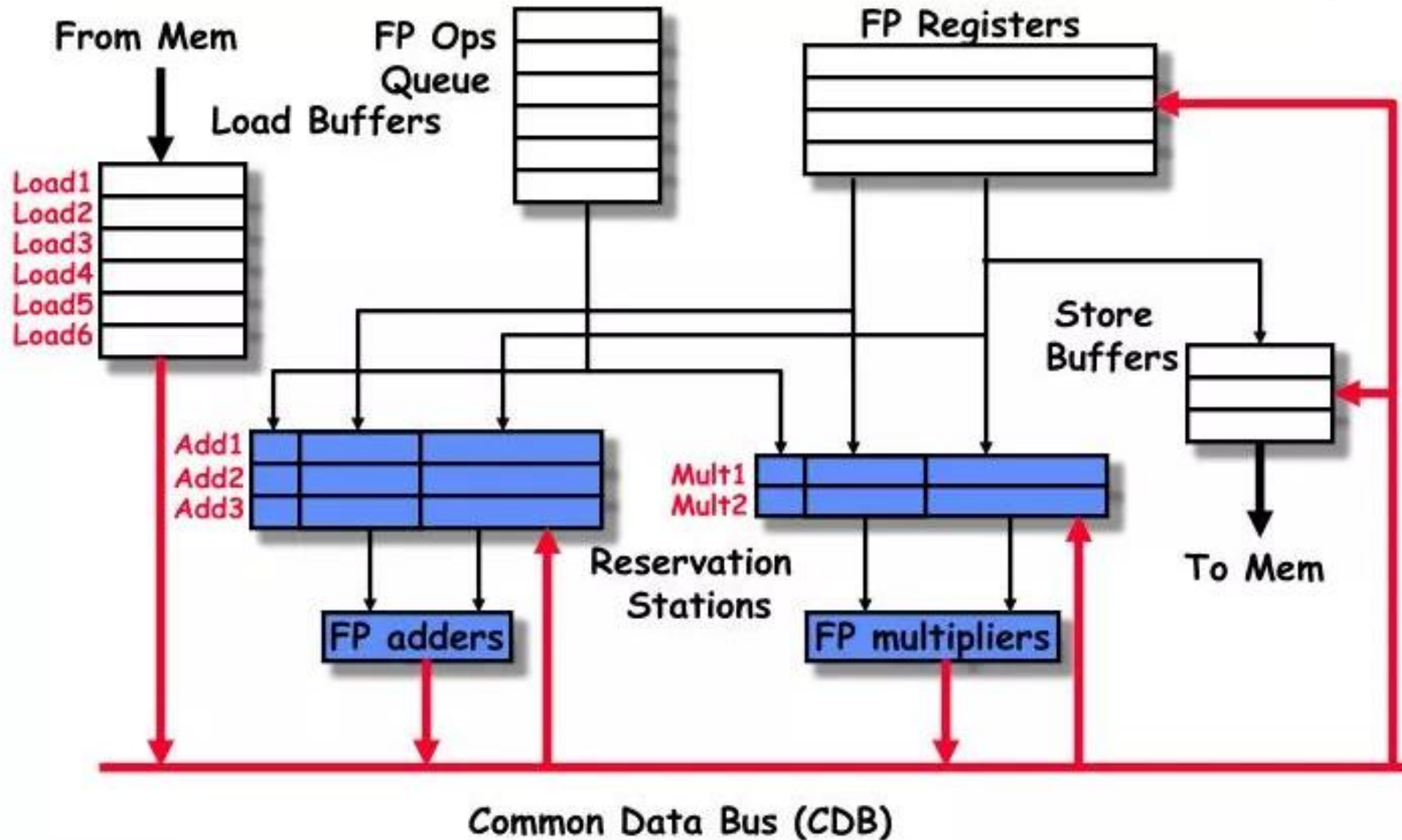
Organizations of Tomasulo's Algorithm in RISC-V



- Load/Store buffer
- Reservation station
- Common data bus

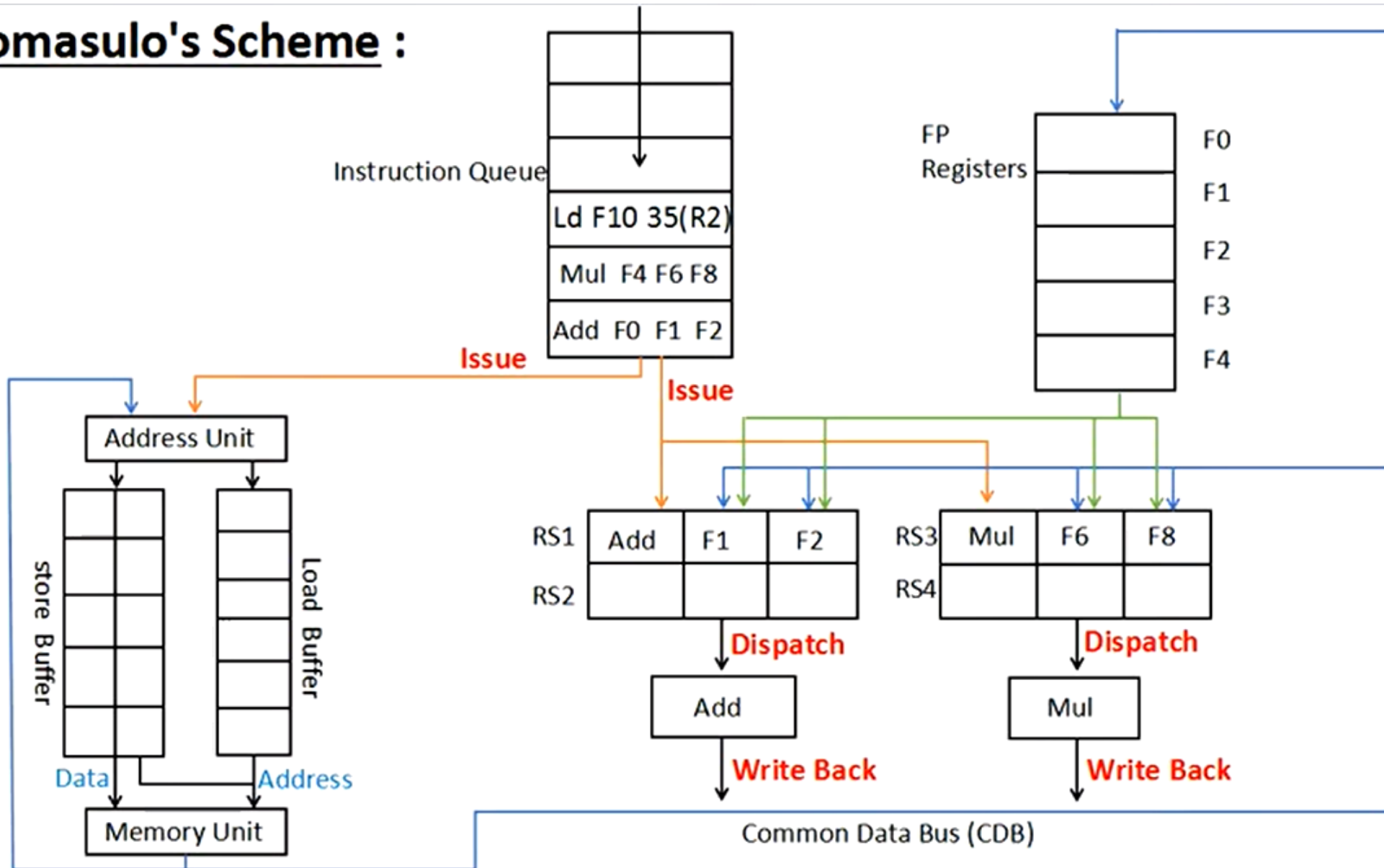


Tomasulo Organization – clear diagram

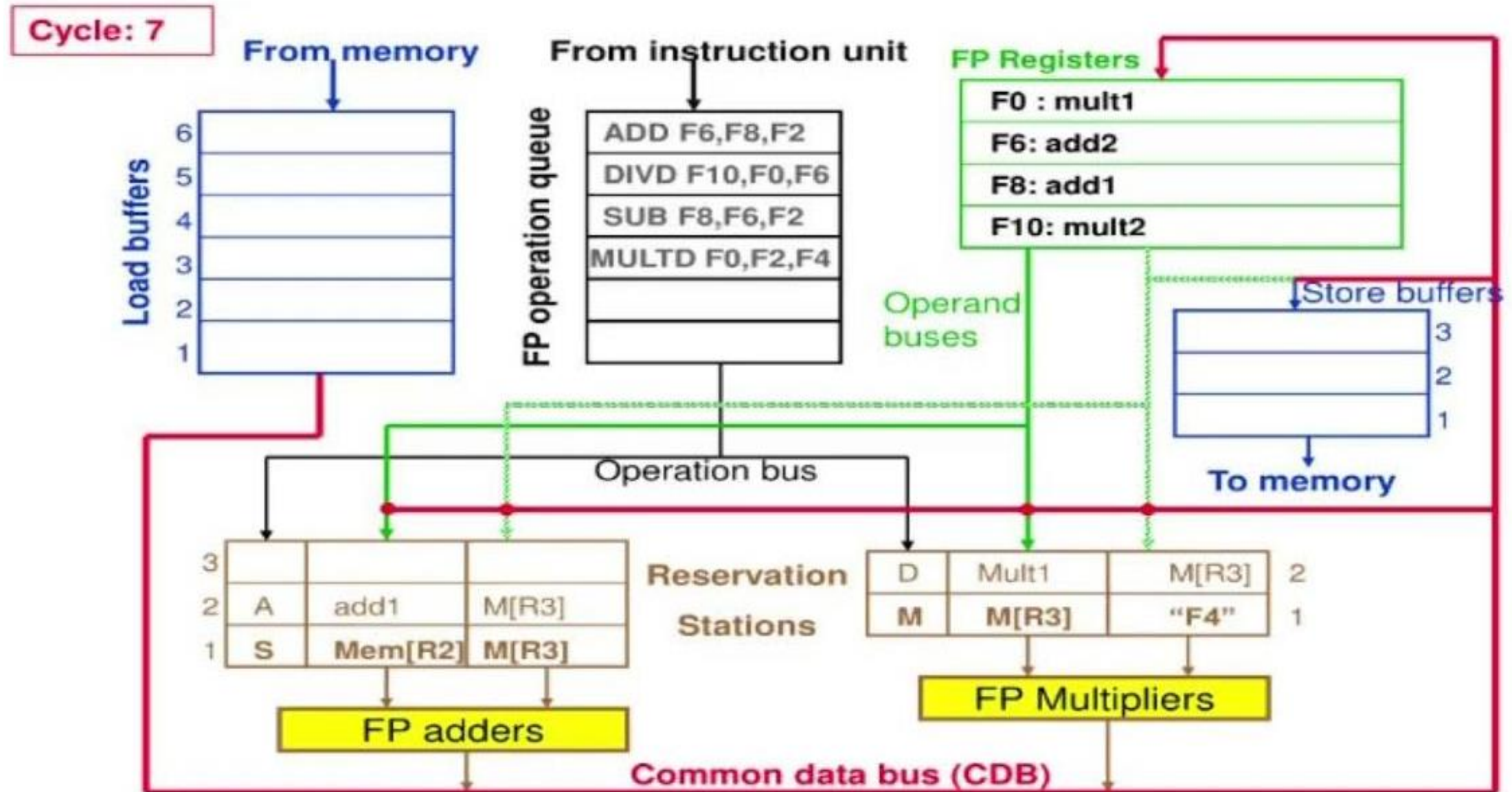


Simplified Tomasulo – with some more labels

Tomasulo's Scheme :



Another Colour view of Tomasulo



3 Parts of Tomasulo

3 Parts Of Tomasulo's Scheme :

1. Instruction Status : In which of the 3 stages(Issue,EX,WB) the instruction is.

2. Reservation Stations Status :

- i. **Busy** : Whether the reservation unit and its accompanying functional unit is busy or not.
- ii. **Op** : Operation to be performed.
- iii. **Q_j, Q_k** : Functional Units producing source operands.
- iv. **V_j, V_k** : Value of the source operands.

3. Register Result Status : Which functional unit is going to write the register.

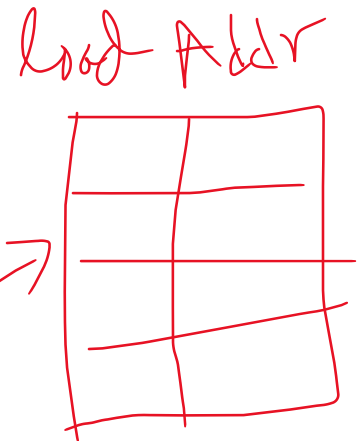
Data Structure of Tumasulo Scheduling

Data Structure

- Qj, Qk: Rsrv. Station for Src1 and Src2
- Vj, Vk: Values of two operands. Values are valid if Qj and Qk are zero
- Busy: Rsrv. Station and FU are busy
- Opcode: Operation to be performed
- A: Memory Addresses for LD/SD (initially Imm Value)
- Qi: Contained in Reg-file, indicates the src Rsrv. Station (like scoreboard)

Is	Ex	W

	Reservation Stations						
	Busy	Op	Vj	Vk	Qj	Qk	A
LD1							
LD2							
AD1							
AD2							
AD3							
ML1							
ML2							
SD1							
SD2							
	F0	F2	F4	F6	F8	F10	F12
Register Status: Qi							...



Question on Tomasulo

Q. Write the steps of execution using Tomasulo's Algorithm. Assume we have 3 loader, 3 adders, 2 multiplier. The following set of MIPS instruction is going to be executed in a pipelined system.

- Reference:

- <https://www.youtube.com/watch?v=EfwCfhXxNnI&list=WL&index=6>

LD F6, 34(R2)
LD F2, 45(R3)
MUL F0, F2, F4
SUB F8, F6, F2
DIV F10, F0, F6
ADD F6, F8, F2

The latencies of - Load instruction is 2 cycle
Add instruction is 2 cycle
Multiply instruction is 10 cycle
Divide instruction is 40 cycles

Blank – Data Structure of Tomasulo

Instruction Status

Instr	Issue	EX	WB

The latencies of - Load instruction is 2 cycle
 Add instruction is 2 cycle
 Multiply instruction is 10 cycle
 Divide instruction is 40 cycles

<http://www.flashbackrecorder.com/>
 Watermark removed when fully licensed

	Busy	Address
Load1		
Load2		
Load3		

Reservation Station Status

Name	Busy	Op	V_j	V_k	Q_j	Q_k
Add1						
Add2						
Add3						
Mult1						
Mult2						

Register Result Status

	F0	F2	F4	F6	F8	F10		F30
FU								

Important Parts of Tomasulo – for Solution

Instruction Status

Instr	Issue	EX	WB
LD F6, 34(R2)			
LD F2, 45(R3)			
MUL F0, F2, F4			
SUB F8, F6, F2			
DIV F10, F0, F6			
ADD F6, F8, F2			

The latencies of - Load instruction is 2 cycle
 Add instruction is 2 cycle
 Multiply instruction is 10 cycle
 Divide instruction is 40 cycles

<http://www.flashbackrecorder.com/>

Watermark removed when fully licensed

	Busy	Address
Load1		
Load2		
Load3		

Reservation Station Status

Name	Busy	Op	V_j	V_k	Q_j	Q_k
Add1						
Add2						
Add3						
Mult1						
Mult2						

Register Result Status

	F0	F2	F4	F6	F8	F10		F30
FU								

Cycle 1

Instruction Status

Instr	Issue	EX	WB
LD F6, 34(R2)	1	I	
LD F2, 45(R3)			
MUL F0, F2, F4			
SUB F8, F6, F2			
DIV F10, F0, F6			
ADD F6, F8, F2			

The latencies of - Load instruction is 2 cycle
 Add instruction is 2 cycle
 Multiply instruction is 10 cycle
 Divide instruction is 40 cycles

<http://www.flashbackrecorder.com/>
 Watermark removed when fully licensed

	Busy	Address
Load1	yes	34+R2
Load2		
Load3		

Reservation Station Status

Name	Busy	Op	V_j	V_k	Q_j	Q_k
Add1						
Add2						
Add3						
Mult1						
Mult2						

Register Result Status

	F0	F2	F4	F6	F8	F10		F30
FU				Load1				

Cycle 2

Instruction Status

Instr	Issue	EX	WB
LD F6, 34(R2)	1	2-	
LD F2, 45(R3)	2		
MUL F0, F2, F4			
SUB F8, F6, F2			
DIV F10, F0, F6			
ADD F6, F8, F2			

	Busy	Address
Load1	yes	34+R2
Load2	Yes	45+R3
Load3		

Reservation Station Status

Name	Busy	Op	V_j	V_k	Q_j	Q_k
Add1						
Add2						
Add3						
Mult1						
Mult2						

Register Result Status

	F0	F2	F4	F6	F8	F10		F30
FU		Load2		Load1				

Cycle 3

Instruction Status

Instr	Issue	EX	WB
LD F6, 34(R2)	1	2-3	
LD F2, 45(R3)	2	3-	
MUL F0, F2, F4	3		
SUB F8, F6, F2			
DIV F10, F0, F6			
ADD F6, F8, F2			

http://www.flashbackrecorder.com
Watermark removed when fully licensed

	Busy	Address
Load1	yes	34+R2
Load2	Yes	45+R3
Load3		

Reservation Station Status

Name	Busy	Op	V_j	V_k	Q_j	Q_k
Add1						
Add2						
Add3						
Mult1	Yes	Mult		R(F4)	Load2	
Mult2			I			

Register Result Status

	F0	F2	F4	F6	F8	F10		F30
FU	Mult1	Load2		Load1				

Cycle 4

Instruction Status

Instr	Issue	EX	WB
LD F6, 34(R2)	1	2-3	4
LD F2, 45(R3)	2	3-4	I
MUL F0, F2, F4	3		
SUB F8, F6, F2	4		
DIV F10, F0, F6			
ADD F6, F8, F2			

	Busy	Address
Load1		
Load2	Yes	45+R3
Load3		

Reservation Station Status

Name	Busy	Op	V _j	V _k	Q _j	Q _k
Add1	Yes	Sub	M(A1)			Load2
Add2						
Add3						
Mult1	Yes	Mult		R(F4)	Load2	
Mult2						

Register Result Status

	F0	F2	F4	F6	F8	F10		F30
FU	Mult1	Load2		M(A1)	Add1			

Cycle 5

Instruction Status

Instr	Issue	EX	WB
LD F6, 34(R2)	1	2-3	4
LD F2, 45(R3)	2	3-4	5
MUL F0, F2, F4	3		
SUB F8, F6, F2	4		
DIV F10, F0, F6	5		
ADD F6, F8, F2	I		

<http://www.flashbackrecorder.com/>
Watermark removed when fully licensed

	Busy	Address
Load1		
Load2		
Load3		

Reservation Station Status

Name	Busy	Op	V_j	V_k	Q_j	Q_k
Add1	Yes	Sub	M(A1)	M(A2)		
Add2						
Add3						
Mult1	Yes	Mult	M(A2)	R(F4)		
Mult2	Yes	Div		M(A2)	Add1	

Register Result Status

	F0	F2	F4	F6	F8	F10		F30
FU	Mult1	M(A2)		M(A1)	Add1	Mult2		

Cycle 6

Instruction Status

Instr	Issue	EX	WB
LD F6, 34(R2)	1	2-3	4
LD F2, 45(R3)	2	3-4	5
MUL F0, F2, F4	3	6-	
SUB F8, F6, F2	4	6-7	8
DIV F10, F0, F6	5		
ADD F6, F8, F2	6		

http://www.flashbackrecorder.com
Watermark removed when fully licensed

	Busy	Address
Load1		
Load2		
Load3		

Reservation Station Status

Name	Busy	Op	V _j	V _k	Q _j	Q _k
Add1						
Add2	Yes	Add		M(A2)	Add1	
Add3						
Mult1	Yes	Mult	M(A2)	R(F4)		
Mult2	Yes	Div		M(A2)	Add1	

Register Result Status

	F0	F2	F4	F6	F8	F10		F30
FU	Mult1	M(A2)		Add2	M1-M2	Mult2		

Cycle 7 to 11

Instruction Status

Instr	Issue	EX	WB
LD F6, 34(R2)	1	2-3	4
LD F2, 45(R3)	2	3-4	5
MUL F0, F2, F4	3	6-	
SUB F8, F6, F2	4	6-7	8
DIV F10, F0, F6	5	I	
ADD F6, F8, F2	6	9-10	11

	Busy	Address
Load1		
Load2		
Load3		

Reservation Station Status

Name	Busy	Op	V_j	V_k	Q_j	Q_k
Add1						
Add2						
Add3						
Mult1	Yes	Mult	M(A2)	R(F4)		
Mult2	Yes	Div		M(A2)	Mult1	

Register Result Status

	F0	F2	F4	F6	F8	F10		F30
FU	Mult1	M(A2)		M1-M2+M(A2)	M1-M2	Mult2		

Cycle 12 to 16

Instruction Status

Instr	Issue	EX	WB
LD F6, 34(R2)	1	2-3	4
LD F2, 45(R3)	2	3-4	5
MUL FO, F2, F4	3	6-15	16
SUB F8, F6, F2	4	6-7	8
DIV F10, F0, F6	5	I	
ADD F6, F8, F2	6	9-10	11

<http://www.flashbackrecorder.com/>
Watermark removed when fully licensed

	Busy	Address
Load1		
Load2		
Load3		

Reservation Station Status

Name	Busy	Op	V_j	V_k	Q_j	Q_k
Add1						
Add2						
Add3						
Mult1						
Mult2	Yes	Div	$M(A2)*R$	$M(A2)$		

Register Result Status

	F0	F2	F4	F6	F8	F10		F30
FU	$M(A2)*R$	$M(A2)$		$M1-M2+M(A2)$	$M1-M2$	Mult2		

Cycle 12 to 57

Instruction Status

Instr	Issue	EX	WB
LD F6, 34(R2)	1	2-3	4
LD F2, 45(R3)	2	3-4	5
MUL F0, F2, F4	3	6-15	16
SUB F8, F6, F2	4	6-7	8
DIV F10, F0, F6	5	17- 56	57
ADD F6, F8, F2	6	9-10	11

<http://www.flashbackrecorder.com/>

Watermark removed when fully licensed

	Busy	Address
Load1		
Load2		
Load3		

Reservation Station Status

Name	Busy	Op	V_j	V_k	Q_j	Q_k
Add1						
Add2						
Add3						
Mult1						
Mult2						

Register Result Status

	F0	F2	F4	F6	F8	F10		F30
FU	$M(A2)*R$	$M(A2)$		$M1-M2+M(A2)$	$M1-M2$	T		

Scoreboard Example

Reference:

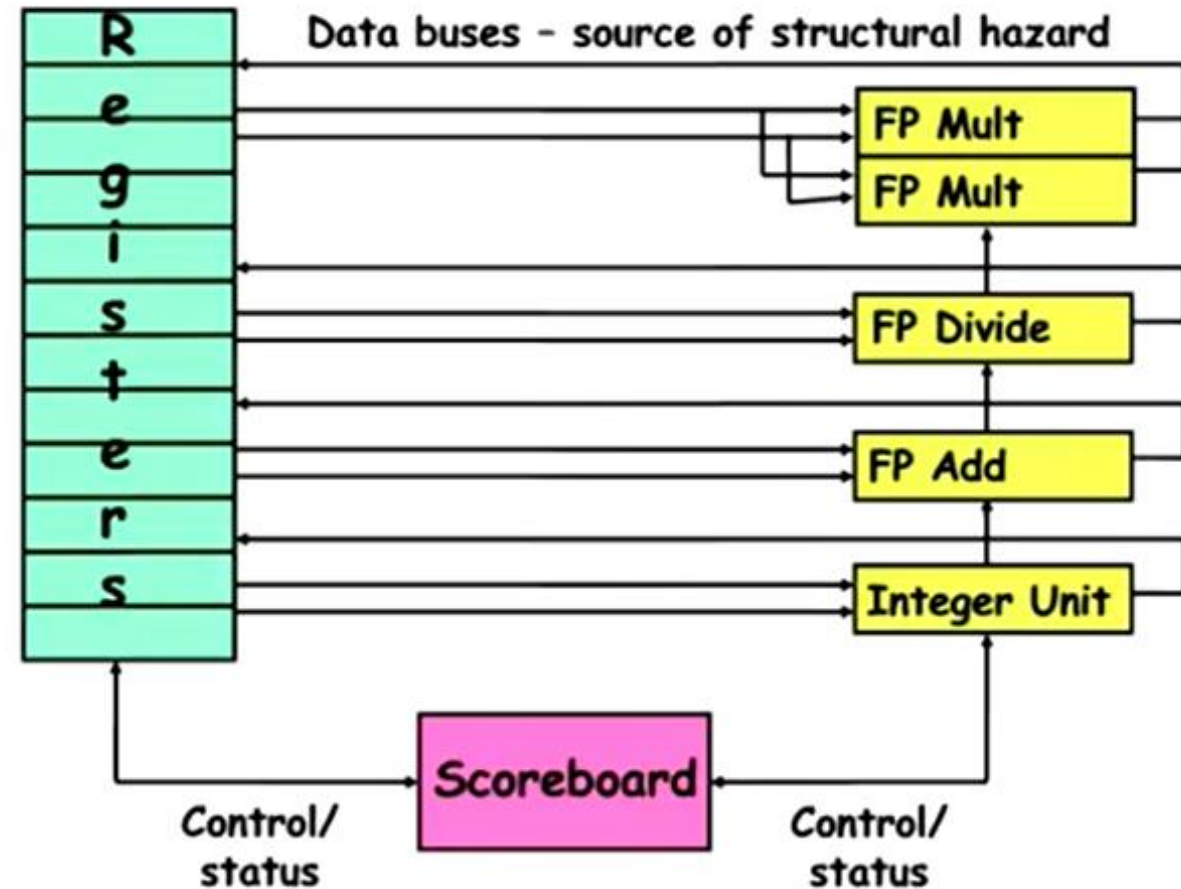
<https://www.youtube.com/watch?v=TJcNJh8tS6w>

A Scoreboard For MIPS :

Every instruction goes through a centralised hardware scoreboard. The scoreboard sends control signal to tell when an instruction can execute and when it can write results into destination register.

Out of order execution requires multiple instruction in the EX stage simultaneously which can be achieved with multiple functional units.

The instructions having a dependence are stalled but those having no dependence are allowed to continue.



3 Parts Of Scoreboard :

1. Instruction Status : In which of the 4 stages(Issue,RO,EX,WB) the instruction is.

2. Functional Unit Status :

- i. Busy : Whether the functional unit is busy or not.
- ii. Op : Operation to be performed.
- iii. F_i : Destination Register.
- iv. F_j, F_k : Source registers.
- v. Q_j, Q_k : Functional unit going to produce source registers F_j and F_k
- vi. R_j, R_k : Flags indicating whether F_j and F_k are ready or not.

3. Register Result Status : Indicates which functional unit will write the registers.

4 Stages of Instruction Scoreboard Control

4 Stages of Scoreboard Control :

Instruction

IS

1. (ID Stage 1) Issue : Here instruction is decoded. If the functional unit is free (ie. no structural hazard), and no previously stalled instruction has the same destination register (ie. no WAW dependency), the scoreboard issues the instruction.

Issue of instruction always takes place in order . If there is structural or WAW hazard then the instruction issue stalls and no further instruction will be issued until these hazards are cleared.

RO

2. (ID Stage 2) Read Operand : A source operand is available or we can read a source operand if no earlier issued stalled instructions is going to write it (ie. the scoreboard resolves RAW hazard dynamically in this stage). When the source operand is available, the scoreboard tells the functional units to proceed to read operands from the registers and begin execution. The instructions may be sent to execution stage out of order.

EX

3. Execution (EX) : Operate on operands. When result is ready control signal is sent to the scoreboard.

WB

4. Write Back (WB) : Once the scoreboard is aware that the functional unit has completed its execution, it checks for WAR hazard. If none, it writes the result. If WAR hazard exists, then it stalls the instruction.

Eg : Div F0 F2 F4
Add F6 F2 F8
Sub F8 F10 F12

Let EX stage of Sub instruction is completed before Add instruction, but to resolve WAR hazard for F8, the sub instruction will be stalled until Add instruction does not read the data from F8.

Instruction Status		
Instruction	j	k

Read	Execution	Write	
Issue	Operand	Complete	Result

Functional Unit Status									
Name	Busy	Op	Fi	Fj	Fk	Qj	Qk	Rj	Rk
Integer									
Mult1									
Mult2									
Add									
Divide									

Register Result Status											
CLOCK		F0	F2	F4	F6	F8	F10	F12	...	F31	
	FU										

Example - Data Structure of Scoreboard Scheduling

Instruction Status				Read	Execution	Write	
Instruction		j	k	Issue	Operand	Complete	Result
LD	F6	34+	R2				
LD	F2	45+	R3				
MULT	F0	F2	F4				
SUBD	F8	F6	F2				
DIVD	F10	F0	F6				
ADDD	F6	F8	F2				

Functional Unit Status

Name	Busy	Op	Fi	Fj	Fk	Qj	Qk	Rj	Rk
Integer	No								
Mult1	No								
Mult2	No								
Add	No								
Divide	No								

Register Result Status

CLOCK	F0	F2	F4	F6	F8	F10	F12	...	F31
0	FU								

Question on Scoreboard



Q. Write the steps of execution with scoreboard approach. The following set of MIPS instruction is going to be executed in a pipelined system.

LD F6, 34(R2)
LD F2, 45(R3)
MUL F0, F2, F4
SUB F8, F6, F2
DIV F10, F0, F6
ADD F6, F8, F2

- Reference:
- <https://www.youtube.com/watch?v=TJcNJh8tS6w>

The latencies of - Integer unit is 1 cycle
 FP adder unit is 2 cycle
 FP multiplier is 10 cycle
 FP divider unit is 40 cycles

Starting with 1st Instruction

1. Instruction Status :

Instr ⁿ . i j k	Issue	RO	EX	WB
LD F6, 34(R2)				
LD F2, 45(R3)				
MUL F0, F2, F4				
SUB F8, F6, F2				
DIV F10, F0, F6				
ADD F6, F8, F2				

1. **Instruction Status** : In which of the 4 stages(Issue,RO,EX,WB) the instruction is.

2. Functional Unit Status :

- i. Busy :Whether the functional unit is busy or not.
- ii. Op : Operation to be performed.
- iii. F_i : Destination Register.
- iv. F_j, F_k : Source registers.
- v. Q_j, Q_k : Functional unit going to produce source registers F_j and F_k
- vi. R_j, R_k : Flags indicating whether F_j and F_k are ready or not.

3. **Register Result Status** : Indicates which functional unit will write the registers.

3. Register Result Status :

	F0	F2	F4	F6	F8	F10	F12	F14
FU								

2. Functional Unit Status :

FU	Busy	Op	F_i	F_j	F_k	Q_j	Q_k	R_j	R_k
Integer									
Mult1									
Mult2									
Add									
Div									

Cycle 1

1. Instruction Status :

Instr ⁿ . i j k	Issue	RO	EX	WB
LD F6, 34(R2)	1			
LD F2, 45(R3)				
MUL F0, F2, F4				
SUB F8, F6, F2				
DIV F10, F0, F6				
ADD F6, F8, F2				

<http://www.flashbackrecorder.com/>
Watermark removed when fully licensed

3. Register Result Status :

	F0	F2	F4	F6	F8	F10	F12	F14
FU				I				

2. Functional Unit Status :

FU	Busy	Op	F _i	F _j	F _k	Q _j	Q _k	R _j	R _k
Integer	Yes	Load	F6		R2				Yes
Mult1									
Mult2									
Add									
Div									

1. Instruction Status :

Instr ⁿ . i j k	Issue	RO	EX	WB
LD F6, 34(R2)	1	2	3	4
LD F2, 45(R3)	I			
MUL F0, F2, F4				
SUB F8, F6, F2				
DIV F10, F0, F6				
ADD F6, F8, F2				

<http://www.flashbackrecorder.com/>
Watermark removed when fully licensed

3. Register Result Status :

	F0	F2	F4	F6	F8	F10	F12	F14
FU								

2. Functional Unit Status :

FU	Busy	Op	F _i	F _j	F _k	Q _j	Q _k	R _j	R _k
Integer									
Mult1									
Mult2									
Add									
Div									

1. Instruction Status :

Instr ⁿ . i j k	Issue	RO	EX	WB
LD F6, 34(R2)	1	2	3	4
LD F2, 45(R3)	5	I		
MUL F0, F2, F4				
SUB F8, F6, F2				
DIV F10, F0, F6				
ADD F6, F8, F2				

The latencies of - Integer unit is 1 cycle
 FP adder unit is 2 cycle
 FP multiplier is 10 cycle
 FP divider unit is 40 cycles

3. Register Result Status :

	F0	F2	F4	F6	F8	F10	F12	F14
FU		Int						

2. Functional Unit Status :

FU	Busy	Op	F _i	F _j	F _k	Q _j	Q _k	R _j	R _k
Integer	Yes	Load	F2		R3				Yes
Mult1									
Mult2									
Add									
Div									

1. Instruction Status :

Instr ⁿ . i j k	Issue	RO	EX	WB
LD F6, 34(R2)	1	2	3	4
LD F2, 45(R3)	5	6	I	
MUL F0, F2, F4	6			
SUB F8, F6, F2				
DIV F10, F0, F6				
ADD F6, F8, F2				

3. Register Result Status :

	F0	F2	F4	F6	F8	F10	F12	F14
FU	Mult1	Int						

2. Functional Unit Status :

FU	Busy	Op	F _i	F _j	F _k	Q _j	Q _k	R _j	R _k
Integer	Yes	Load	F2		R3				Yes
Mult1	Yes	Mult	F0	F2	F4	Int		No	Yes
Mult2									
Add									
Div									

1. Instruction Status :

Instr ⁿ . i j k	Issue	RO	EX	WB
LD F6, 34(R2)	1	2	3	4
LD F2, 45(R3)	5	6	7	
MUL F0, F2, F4	6			
SUB F8, F6, F2	7		I	
DIV F10, F0, F6				
ADD F6, F8, F2				

<http://www.flashbackrecorder.com/>
Watermark removed when fully licensed

3. Register Result Status :

	F0	F2	F4	F6	F8	F10	F12	F14
FU	Mult1	Int			Add			

2. Functional Unit Status :

FU	Busy	Op	F _i	F _j	F _k	Q _j	Q _k	R _j	R _k
Integer	Yes	Load	F2		R3				Yes
Mult1	Yes	Mult	F0	F2	F4	Int		No	Yes
Mult2									
Add	Yes	Sub	F8	F6	F2		Int	Yes	No
Div									

Cycle 8

1. Instruction Status :

Instr ⁿ . i j k	Issue	RO	EX	WB
LD F6, 34(R2)	1	2	3	4
LD F2, 45(R3)	5	6	7	8
MUL F0, F2, F4	6			
SUB F8, F6, F2	7			
DIV F10, F0, F6	8			
ADD F6, F8, F2				

<http://www.flashbackrecorder.com/>
Watermark removed when fully licensed

3. Register Result Status :

	F0	F2	F4	F6	F8	F10	F12	F14
FU	Mult1				Add	Div		

2. Functional Unit Status :

FU	Busy	Op	F _i	F _j	F _k	Q _j	Q _k	R _j	R _k
Integer									
Mult1	Yes	Mult	F0	F2	F4			Yes	Yes
Mult2									
Add	Yes	Sub	F8	F6	F2			Yes	Yes
Div	Yes	Div	F10	F0	F6	Mult1		No	Yes

Cycle 9 to 11

1. Instruction Status :

Instr ⁿ . i j k	Issue	RO	EX	WB
LD F6, 34(R2)	1	2	3	4
LD F2, 45(R3)	5	6	7	8
MUL F0, F2, F4	6	9		
SUB F8, F6, F2	7	9	11	12
DIV F10, F0, F6	8			
ADD F6, F8, F2	I ₃			

3. Register Result Status :

	F0	F2	F4	F6	F8	F10	F12	F14
FU	Mult1					Div		

2. Functional Unit Status :

FU	Busy	Op	F _i	F _j	F _k	Q _j	Q _k	R _j	R _k
Integer									
Mult1	Yes	Mult	F0	F2	F4			Yes	Yes
Mult2									
Add									
Div	Yes	Div	F10	F0	F6	Mult1		No	Yes

1. Instruction Status :

Instr ⁿ . i j k	Issue	RO	EX	WB
LD F6, 34(R2)	1	2	3	4
LD F2, 45(R3)	5	6	7	8
MUL F0, F2, F4	6	9		
SUB F8, F6, F2	7	9	11	12
DIV F10, F0, F6	8			
ADD F6, F8, F2	I			

3. Register Result Status :

	F0	F2	F4	F6	F8	F10	F12	F14
FU	Mult1					Div		

2. Functional Unit Status :

FU	Busy	Op	F _i	F _j	F _k	Q _j	Q _k	R _j	R _k
Integer									
Mult1	Yes	Mult	F0	F2	F4			Yes	Yes
Mult2									
Add									
Div	Yes	Div	F10	F0	F6	Mult1		No	Yes

1. Instruction Status :

Instr ⁿ . i j k	Issue	RO	EX	WB
LD F6, 34(R2)	1	2	3	4
LD F2, 45(R3)	5	6	7	8
MUL F0, F2, F4	6	9		
SUB F8, F6, F2	7	9	11	12
DIV F10, F0, F6	8			
ADD F6, F8, F2	13	I ¹⁴		

3. Register Result Status :

	F0	F2	F4	F6	F8	F10	F12	F14
FU	Mult1			Add		Div		

2. Functional Unit Status :

FU	Busy	Op	F _i	F _j	F _k	Q _j	Q _k	R _j	R _k
Integer									
Mult1	Yes	Mult	F0	F2	F4			Yes	Yes
Mult2									
Add	Yes	Add	F6	F8	F2			Yes	Yes
Div	Yes	Div	F10	F0	F6	Mult1		No	Yes

Cycle 14 to 16

1. Instruction Status :

Instr ⁿ . i j k	Issue	RO	EX	WB
LD F6, 34(R2)	1	2	3	4
LD F2, 45(R3)	5	6	7	8
MUL F0, F2, F4	6	9	19	20
SUB F8, F6, F2	7	9	11	12
DIV F10, F0, F6	8			
ADD F6, F8, F2	13	14	16	

<http://www.flashbackrecorder.com/>
Watermark removed when fully licensed

3. Register Result Status :

	F0	F2	F4	F6	F8	F10	F12	F14
FU				Add		Div		

2. Functional Unit Status :

FU	Busy	Op	F _i	F _j	F _k	Q _j	Q _k	R _j	R _k
Integer									
Mult1	Yes	Mult	F0	F2	F4			Yes	Yes
Mult2									
Add	Yes	Add	F6	F8	F2			Yes	Yes
Div	Yes	Div	F10	F0	F6	Mu		Yes	Yes

Cycle 16 to 22 – avoid WAR, WAW hazards

<http://www.flashbackrecorder.com/>
Watermark removed when fully licensed

1. Instruction Status :

Instr ⁿ . i j k	Issue	RO	EX	WB
LD F6, 34(R2)	1	2	3	4
LD F2, 45(R3)	5	6	7	8
MUL FO, F2, F4	6	9	19	20
SUB F8, F6, F2	7	9	11	12
DIV F10, F0, F6	8	21		
ADD F6, F8, F2	13	14	16	22

3. Register Result Status :

	F0	F2	F4	F6	F8	F10	F12	F14
FU						Div		

2. Functional Unit Status :

FU	Busy	Op	F _i	F _j	F _k	Q _j	Q _k	R _j	R _k
Integer									
Mult1	Yes								
Mult2									
Add									
Div	Yes	Div	F10	F0	F6			Yes	Yes

Cycle 22 to 62

1. Instruction Status :

<http://www.flashbackrecorder.com/>
Watermark removed when fully licensed

Instr ⁿ . i j k	Issue	RO	EX	WB
LD F6, 34(R2)	1	2	3	4
LD F2, 45(R3)	5	6	7	8
MUL F0, F2, F4	6	9	19	20
SUB F8, F6, F2	7	9	11	12
DIV F10, F0, F6	8	21	61	62
ADD F6, F8, F2	13	14	16	22

3. Register Result Status :

	F0	F2	F4	F6	F8	F10	F12	F14
FU								

I

2. Functional Unit Status :

FU	Busy	Op	F _i	F _j	F _k	Q _j	Q _k	R _j	R _k
Integer									
Mult1									
Mult2									
Add									
Div									

Peculiar Aspects of Tumasulo

Details of Reservation Stations & Register Renaming



In Tomasulo's scheme, register renaming is provided by reservation stations, which buffer the operands of instructions waiting to issue and are associated with the functional units. The basic idea is that a reservation station fetches and buffers an operand as soon as it is available, eliminating the need to get the operand from a register. In addition, pending instructions designate the reservation station that will provide their input. Finally, when successive writes to a register overlap in execution, only the last one is actually used to update the register. As instructions are issued, the register specifiers for pending operands are renamed to the names of the reservation station, which provides register renaming.

Because there can be more reservation stations than real registers, the technique can even eliminate hazards arising from name dependences that could not be eliminated by a compiler. As we explore the components of Tomasulo's scheme, we will return to the topic of register renaming and see exactly how the renaming occurs and how it eliminates WAR and WAW hazards.

The use of reservation stations, rather than a centralized register file, leads to two other important properties. First, hazard detection and execution control are distributed: the information held in the reservation stations at each functional unit determines when an instruction can begin execution at that unit. Second, results are passed directly to functional units from the reservation stations where they are buffered, rather than going through the registers. This bypassing is done with a common result bus that allows all units waiting for an operand to be loaded simultaneously (on the 360/91, this is called the common data bus, or CDB). In pipelines that issue multiple instructions per clock and also have multiple execution units, more than one result bus will be needed.

CDB Bus + Reservation Station + Tags



In Tomasulo's scheme, as well as the subsequent methods we look at for supporting speculation, results are broadcast on a bus (the CDB), which is monitored by the reservation stations. The combination of the common result bus and the retrieval of results from the bus by the reservation stations implements the forwarding and bypassing mechanisms used in a statically scheduled pipeline. In doing so, however, a dynamically scheduled scheme, such as Tomasulo's algorithm, introduces one cycle of latency between source and result because the matching of a result and its use cannot be done until the end of the Write Result stage, as opposed to the end of the Execute stage for a simpler pipeline. Thus, in a dynamically scheduled pipeline, the effective latency between a producing instruction and a consuming instruction is at least one cycle longer than the latency of the functional unit producing the result.

forwarding
1 cycle lat

It is important to remember that the tags in the Tomasulo scheme refer to the buffer or unit that will produce a result; the register names are discarded when an instruction issues to a reservation station (This is a key difference between Tomasulo's scheme and scoreboarding: in scoreboarding, operands stay in the registers and are read only after the producing instruction completes and the consuming instruction is ready to execute.)

//

Reading Notes

Readings:
Chapter 3 of course textbook

Also watch youtube examples of Scoreboard and Tomasulo as mentioned later

High Performance Computer Architecture - Scoreboarding Example



HARSHIT ANAND



High Performance Computer Architecture - Tomasulo's Algorithm Example



HARSHIT ANAND



Hardware Based Speculation – Using Reorder Buffer

Re-Order Buffer (ROB)

➤ ROB incorporates two following concerns

RS = Reservation Station

How about the separating register renaming from RS?
(to free RS at an early time!)

It requires finishing the instructions in the program order!

Reorder Buffer – Purpose

- **Hardware Based Speculation:** Earlier, with dynamic scheduling, instructions after the branch were executed only after the branch was resolved. So, during the write-back stage, they wrote into registers or memory.

Now, **with speculation**, we execute instructions along *predicted execution paths*, and the results can only be *committed* if the prediction was correct.

✓ Instruction commit allows an instruction to update the register file or memory only when the instruction is no longer speculative.

- This indicates that we need an additional **piece of hardware that will buffer the results of the instruction till they commit**. This piece of hardware is called the **Reorder Buffer**, abbreviated as **ROB**.

This helps to prevent any irrevocable action until an instruction commits, i.e. updating state or taking an execution.

The **reorder buffer** holds the result of an instruction from the time it executes to the time it **commits**. It is also used to **pass operands among speculated instructions**.

Add Commit Phase to Execution – Reorder Buffer

The key idea behind implementing speculation is to allow instructions to execute out of order but to force them to commit *in order* and to prevent any irrevocable action (such as updating state or taking an exception) until an instruction commits. Therefore, when we add speculation, we need to separate the process of completing execution from instruction commit, because instructions may finish execution considerably before they are ready to commit. Adding this commit phase to the instruction execution sequence requires an additional set of hardware buffers that hold the results of instructions that have finished execution but have not committed. This hardware buffer, which we call the *reorder buffer*, is also used to pass results among instructions that may be speculated.

The reorder buffer (ROB) provides additional registers in the same way as the reservation stations in Tomasulo's algorithm extend the register set. The ROB holds the result of an instruction between the time the operation associated with the instruction completes and the time the instruction commits. The ROB therefore is a source of operands for instructions, just as the reservation stations provide operands in Tomasulo's algorithm. The key difference is that in Tomasulo's algorithm, once an instruction writes its result, all subsequently issued instructions will find the result in the register file. With speculation, the register file is not updated until the instruction commits (and we know definitively that the instruction should execute); thus, the ROB supplies operands in the interval between completion of instruction execution and instruction commit. The ROB is similar to the store buffer in Tomasulo's algorithm, and we integrate the function of the store buffer into the ROB for simplicity.

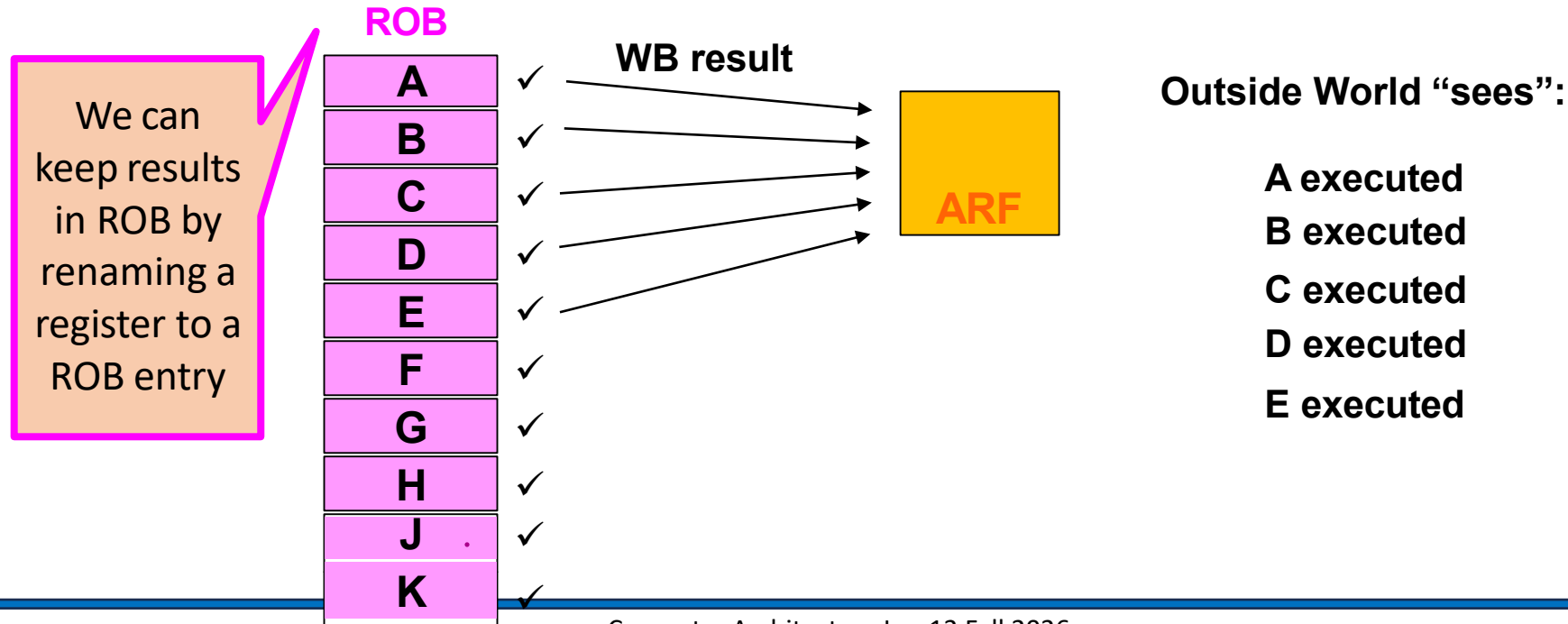
ROB And
Register File

Key Idea of Re-Order Buffer

- **Goal**: let's complete the instructions in the program-order
- **Key idea**: let's execute the instructions out of program order, but just make outside world can **“believe”** it's in-order
- Solution: **Re-Order Buffer** + **Architected Register File**

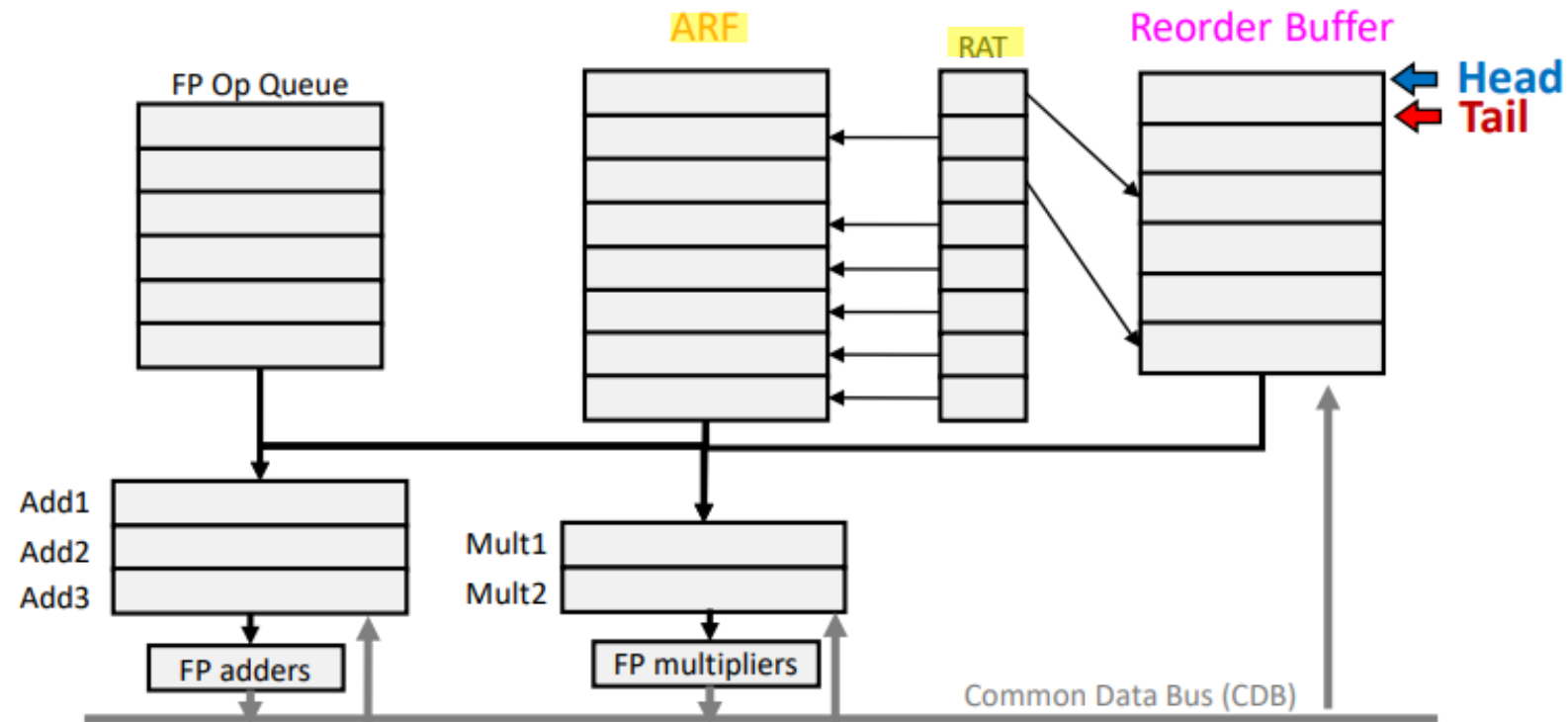
→ Keep the temporal results
(executed in out-of-order)

→ Keep the final results
(illusion of in-order execution)



Tomasulo with ROB Organization

- Re-Order buffer is based on Tomasulo
- Just renamed FP register to **ARF** (Architected Register File)
- Add **Re-Order Buffer** for out-of-order results
 - Buffer is managed with two pointers (head & tail)
- **RAT** (Register Alias Table) keeps the register renaming info.



Four Stages of Tomasulo with ROB - Issue

Stage1: Issue

- Read an instruction from the buffer
- Check if resources available (RS entry & ROB entry)

Stall if any target resource is unavailable

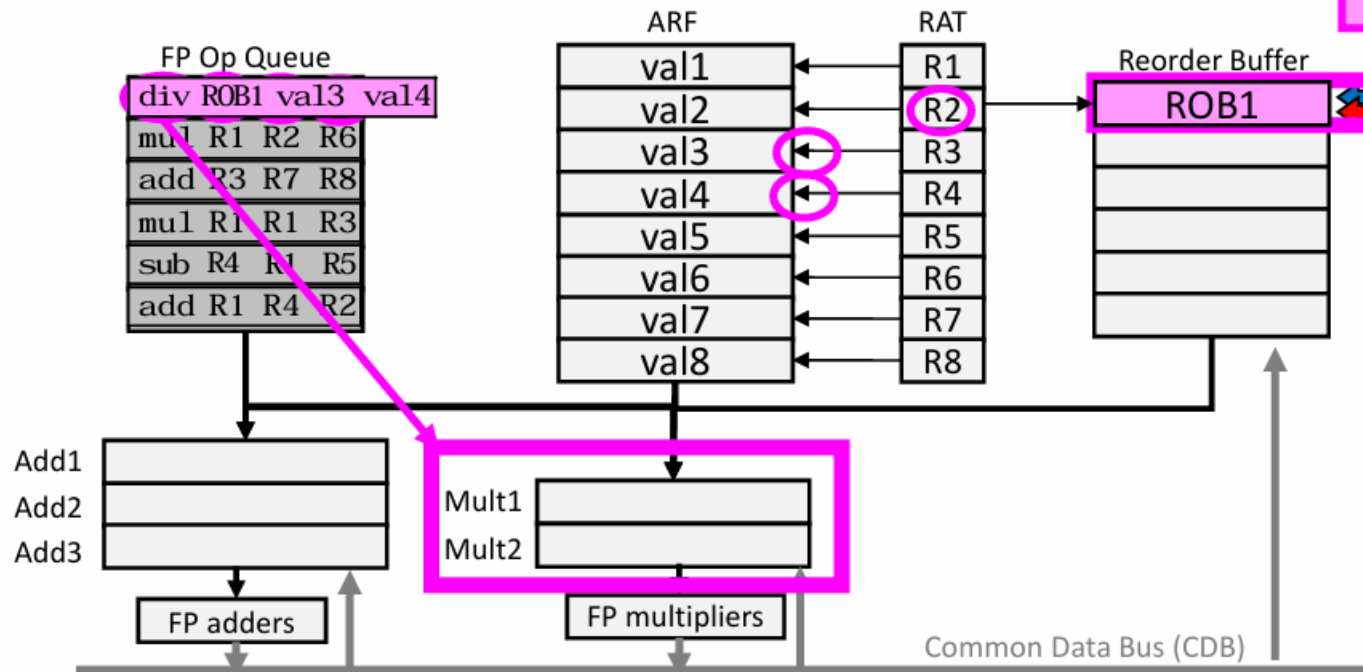
- Read RAT

Sources: read (available) sources

Destination: update RAT

- Write to RS and ROB (the tail of ROB) & update the tail of ROB

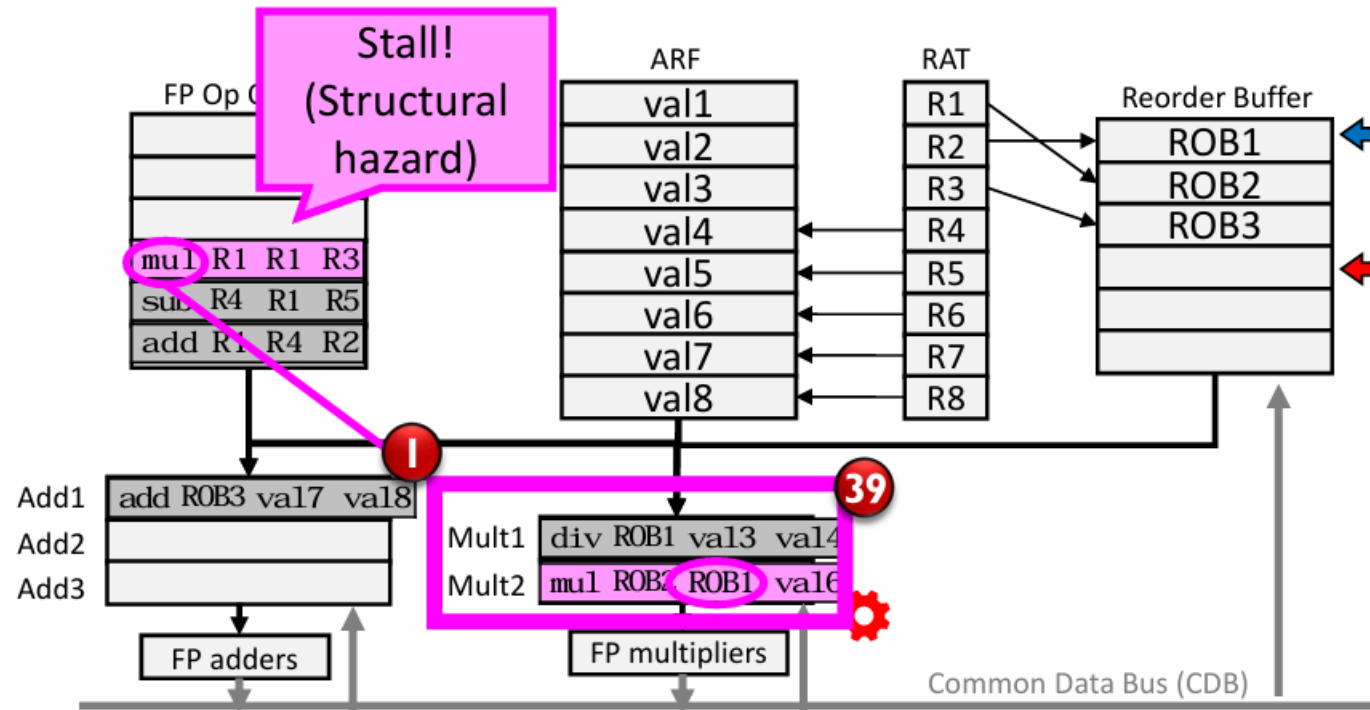
Where the tail pointer indicates



Four Stages of Tomasulo with ROB - Exec

Stage2: Exec

- Different behaviors based on the target instruction status
- For "a not-ready instruction" → Wait for all operands to arrive
 - For "a new instruction" → Compete to use the functional unit
 - For "an all-ready instruction" → Execute!

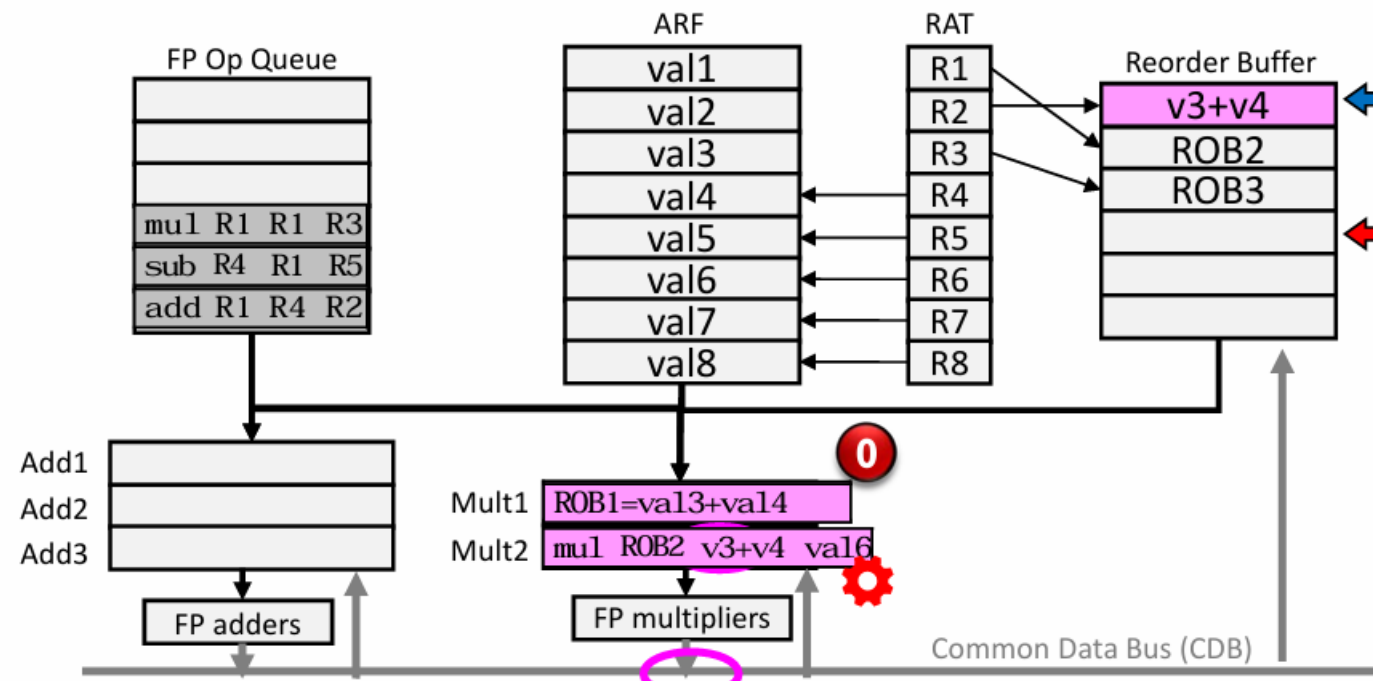


Four Stages of Tomasulo with ROB – Write Result



Stage 3: Write Result

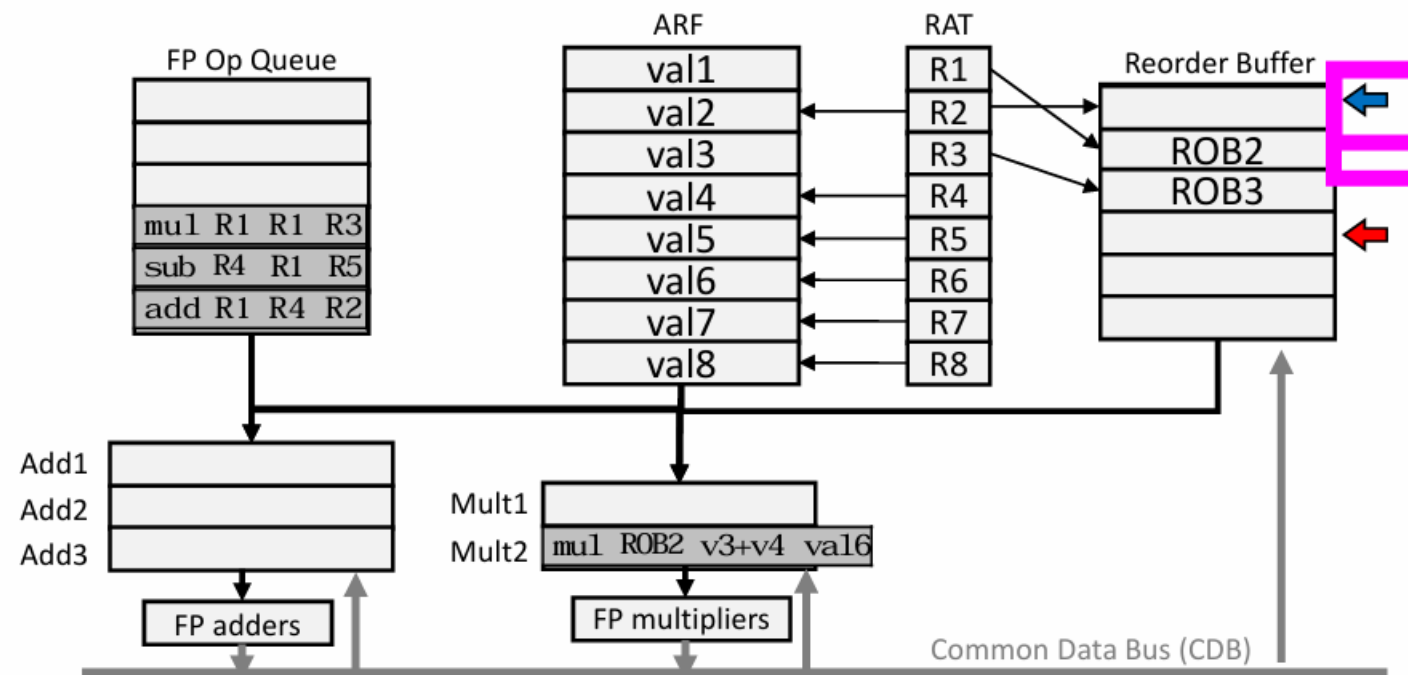
- Broadcast result on CDB
 - (any dependents will grab the value)
- Write result back to your ROB entry
 - The ARF holds the "official" register state, which we will only update in the program order
 - Mark ready/finished bit in ROB (note that this instruction has completed execution)
- Reservation station can be freed



Four Stages of Tomasulo with ROB - Commit

Stage4: Commit

- When the oldest instruction is completed in the ROB
 - i.e., ROB's head points to it
- Write the result (if ready/finished bit is set)
- Advance ROB's head to the next instruction
- This is what the outside world sees
 - And it's all in-order



4 Fields at each ROB location



Each entry in the ROB contains four fields:

- **Instruction Type**

The instruction type field indicates whether the instruction is a branch (and has no destination result), a store (which has a memory address destination), or a register operation (ALU operation or load, which has register destinations).

- **Destination Field**

The destination field supplies the register number (for loads and ALU operations) or the memory address (for stores) where the instruction result should be written.

- **Value Field**

The value field is used to hold the value of the instruction result until the instruction commits.

- **Ready Field**

The ready field indicates that the instruction has completed execution, and the value is ready.

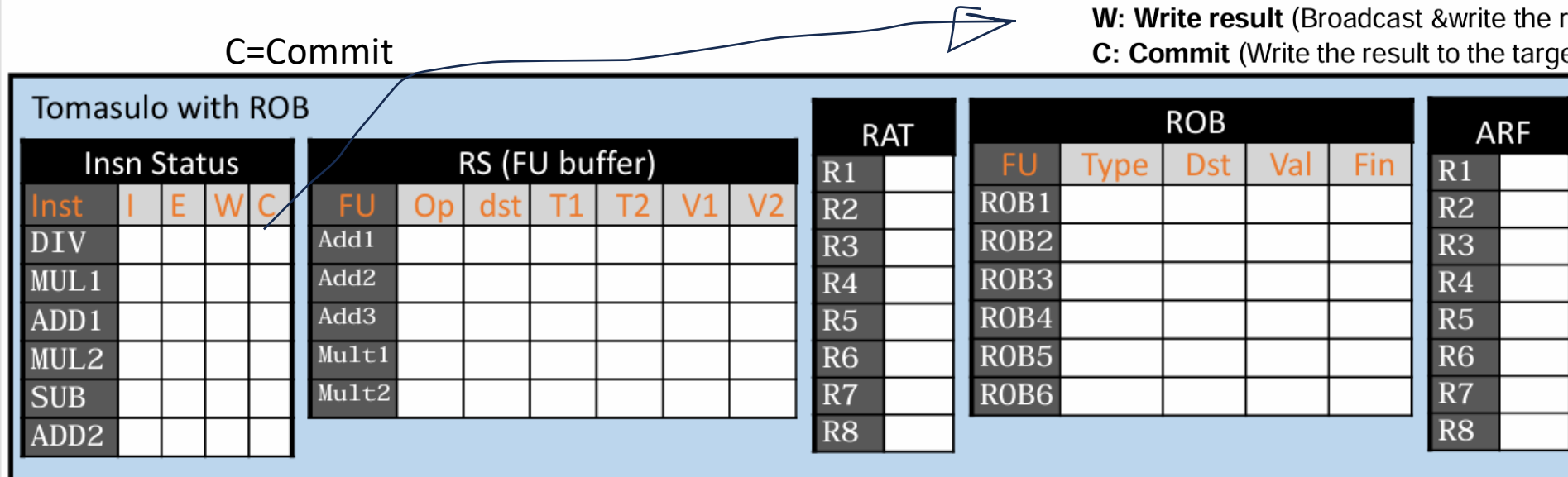
Data Structure of Tomasulo with ROB

➤ Five main components

- Instruction status
- Reservation Stations (FU buffer & Load buffer)
- Register Alias Table (RAT)
- Re-order Buffer (ROB)
- Architected Register File (ARF)

• Instruction status

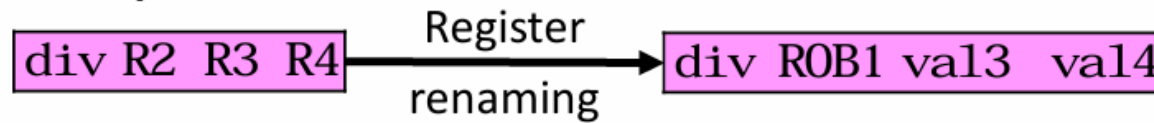
- Which of 4 steps the instruction is in
 - I: Issue** (Read an instruction & check resources availability)
 - E: Execution** (Wait operands / FU or computation)
 - W: Write result** (Broadcast & write the result to ROB & free RS)
 - C: Commit** (Write the result to the target register; ARF)



ReOrder Buffer ROB Operation

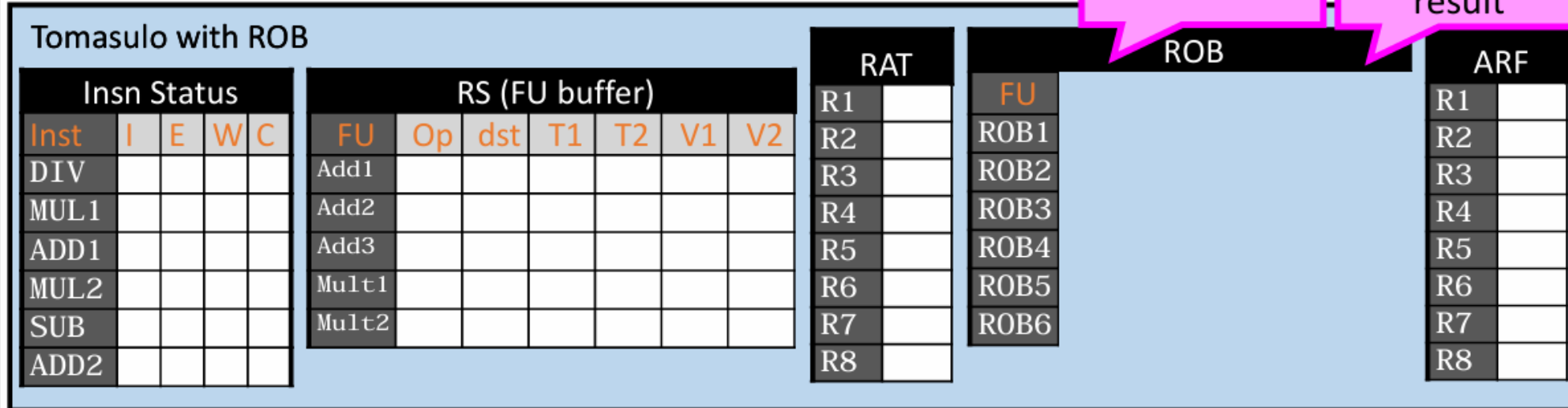
- Re-Order Buffer (Temporally store results)
 - **Type:** Instruction who will update this ROB
 - **Dst:** The original ARF before register renaming
 - **Val:** The result of instruction
 - **Fin:** Ready/finished bit

• Example



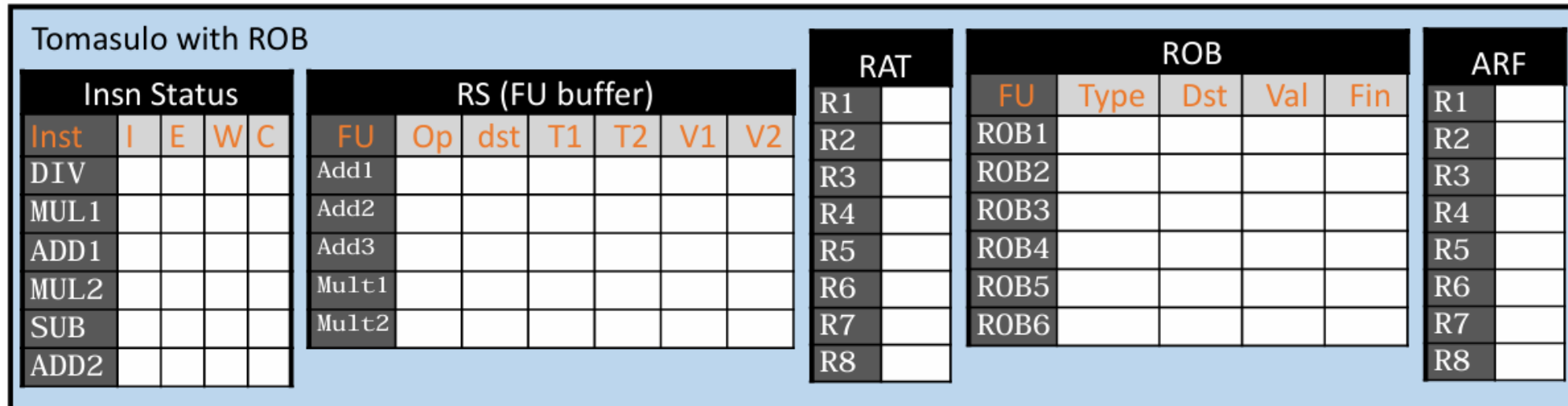
Register renaming

Temporally store instruction result



Architected Register File ARF

- Same with conventional register file of μ processor
 → But the ARF only can be updated with the new pipeline stage "commit"



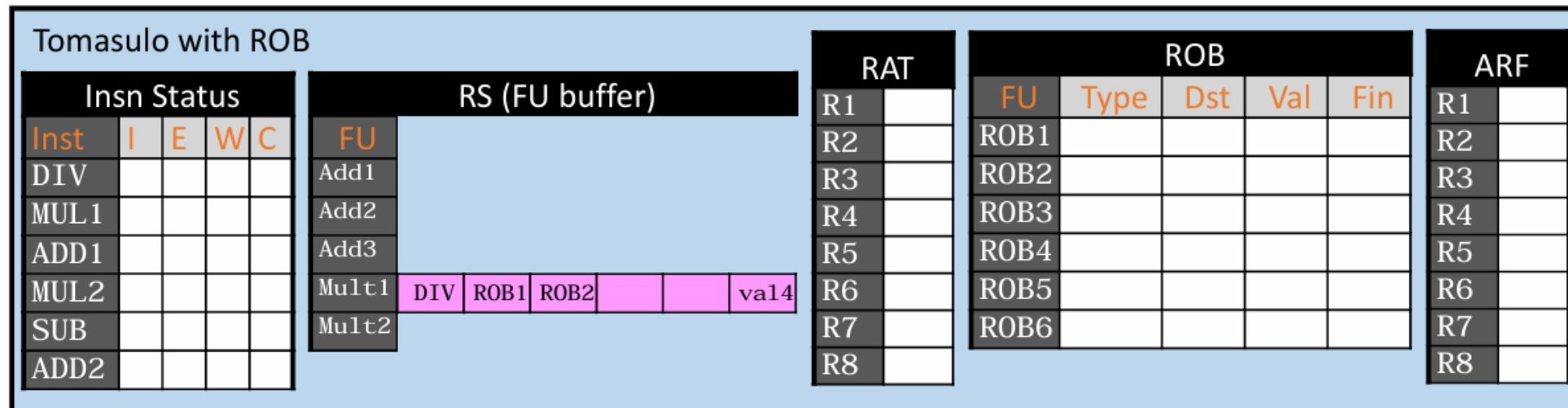
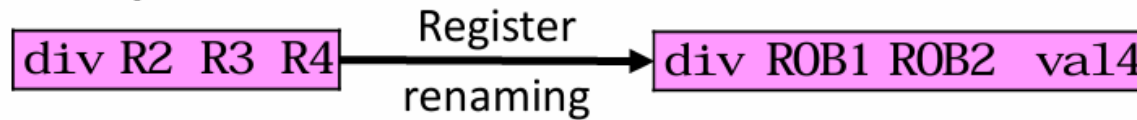
Reservation Stations RS

- FU buffer

- Op: Operation to perform in the unit
- Dst: Tag (ROB) of destination
- T1,T2: Tag (ROB) of source operands
- V1,V2: Value from ARF (Architected Register File)

One of these, should exist!

- Example



Tomasulo with Speculation

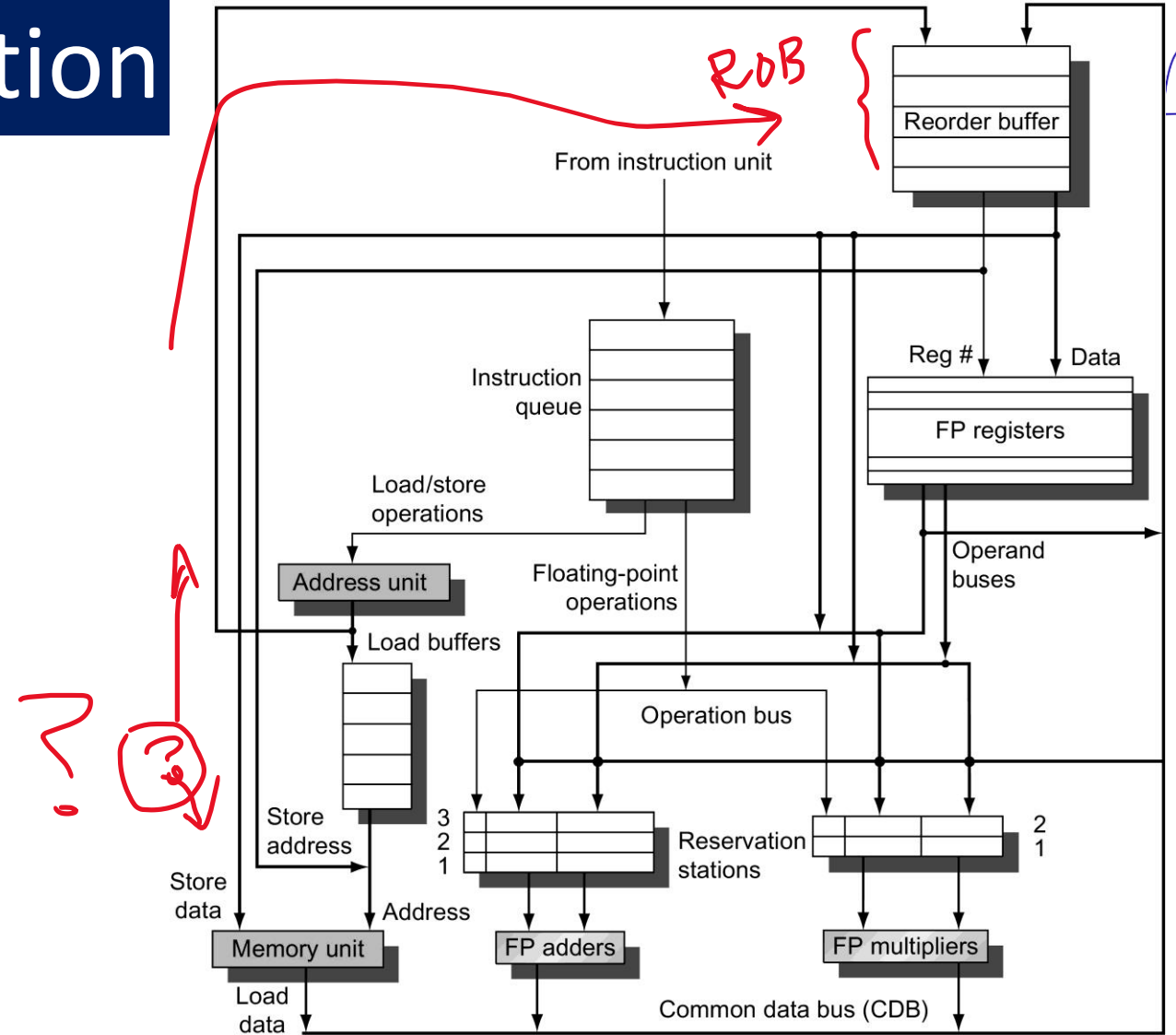


Figure 3.15 The basic structure of a FP unit using Tomasulo's algorithm and extended to handle speculation. Comparing this to Figure 3.10 on page 198, which implemented Tomasulo's algorithm, we can see that the **major change is the addition of the ROB and the elimination of the store buffer, whose function is integrated into the ROB.** This mechanism can be extended to allow multiple issues per clock by making the CDB wider to allow for multiple completions per clock.

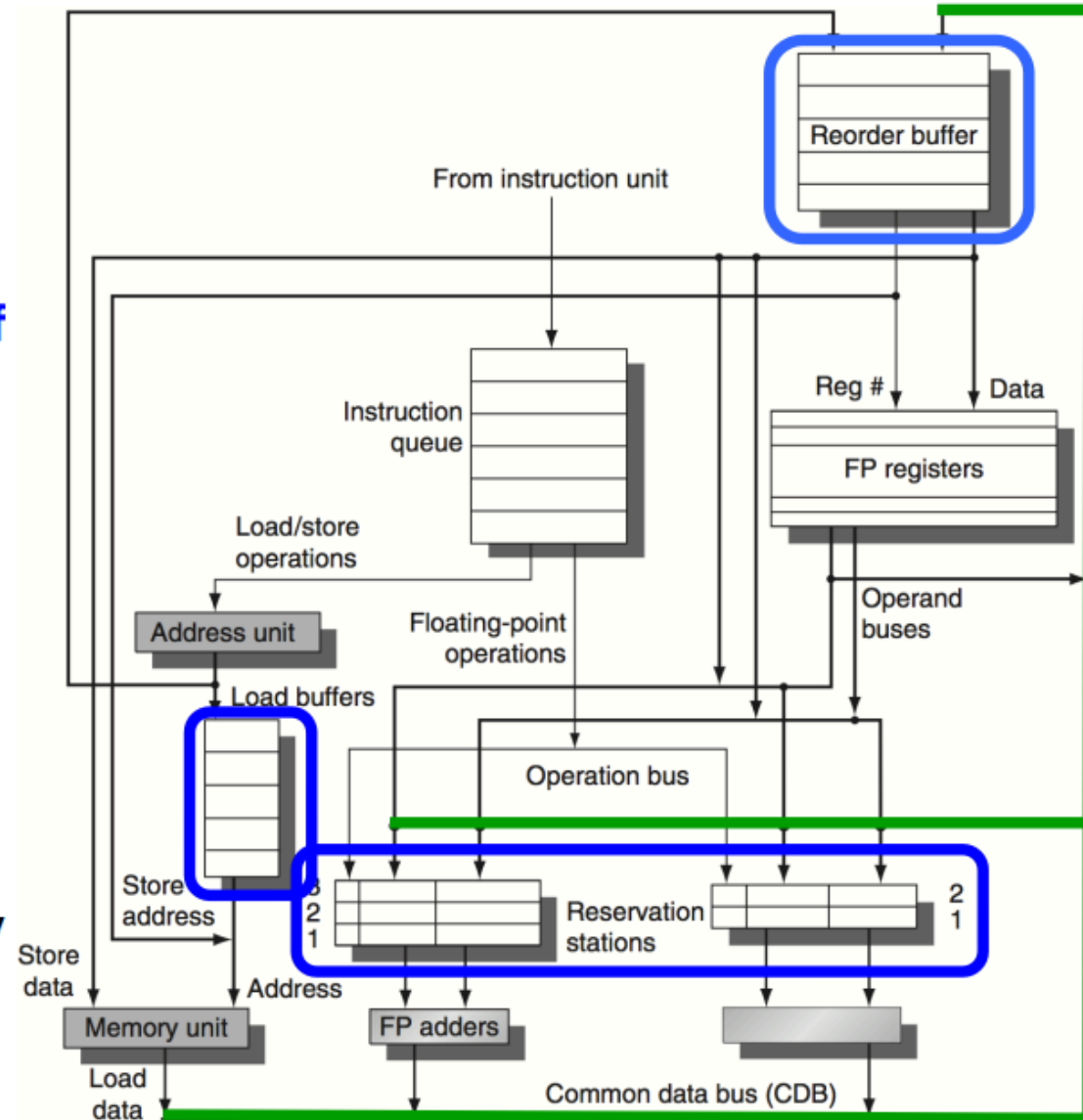
Instruction “in order” Commit

- Also called completion or graduation
- In-order commit
 - In-order issue
 - Out-of-order execution
 - Out-of-order completion
- Three cases when an instr reaches the head of ROB
 - Normal commit: when an instruction reaches the head of the ROB and its result is present in the buffer
 - » The processor updates the register with the result and removes the instruction from the ROB.
 - Committing a store:
 - » is similar except that memory is updated rather than a result register.
 - A branch with incorrect prediction
 - » indicates that the speculation was wrong.
 - » The ROB is flushed and execution is restarted at the correct successor of the branch.

Commit

Hardware Speculation in Tomasulo Algorithm

- **Reservation Station and Load Buffer**
 - Register renaming
 - For dynamic scheduling and out-of order execution
- **Reorder Buffer**
 - Register renaming
 - For in-order commit
- **Common Data Bus**
 - Data forwarding
- **Also handle memory data hazard**



4 Steps of 'Speculation + Tomasulo'



Four Steps of *Speculative* Tomasulo

1. **Issue**—get instruction from FP Op Queue

If reservation station and reorder buffer slot free, issue instr & send operands & reorder buffer no. for destination (this stage sometimes called “dispatch”)

2. **Execution**—operate on operands (EX)

When both operands ready then execute; if not ready, watch CDB for result; when both in reservation station, execute; checks RAW (sometimes called “issue”)

3. **Write result**—finish execution (WB)

Write on Common Data Bus to all awaiting FUs & reorder buffer; mark reservation station available.

4. **Commit**—update register with reorder result

When instr. at head of reorder buffer & result present, update register with result (or store to memory) and remove instr from reorder buffer. Mispredicted branch flushes reorder buffer (sometimes called “graduation”)

Speculative Tomasulo Algorithm

1. **Issue**—get instruction from FP Op Queue
 - ◆ Condition: a free RS at the required FU
 - Actions: (1) **decode** the instruction; (2) **allocate** a RS and ROB entry; (3) do **source register renaming**; (4) do **dest register renaming**; (5) read register file; (6) **issue** the decoded and renamed instruction to the RS and ROB
2. **Execution**—operate on operands (EX)
 - ◆ Condition: At a given FU, At least one instruction is ready
 - ◆ Action: **select** a ready instruction and send it to the FU
3. **Write result**—finish execution (WB)
 - ◆ Condition: At a given FU, some instruction finishes FU execution
 - Actions: (1) FU writes to CDB, broadcast to all RSs **and to the ROB**; (2) FU broadcast tag (ROB index) to all RS; (3) de-allocate the RS. Note: no register status update at this time

Speculative Tomasulo Algorithm

4. **Commit**—update register with reorder result

- ◆ Condition: ROB is not empty and the **ROB head inst** has finished execution
- ◆ Actions if **no mis-prediction/exception**: (1) write result to register/memory, (2) update register status, (3) de-allocate the ROB entry
- ◆ Actions if with **mis-prediction/exception**: **flush the pipeline**, e.g. (1) flush IFQ; (2) clear register status; (3) flush all RS and reset FU; (4) reset ROB

Fields and Store Operation in ROB

Figure 3.15 shows the hardware structure of the processor including the ROB. Each entry in the ROB contains four fields: the instruction type, the destination field, the value field, and the ready field. The instruction type field indicates whether the instruction is a branch (and has no destination result), a store (which has a memory address destination), or a register operation (ALU operation or load, which has register destinations). The destination field supplies the register number (for loads and ALU operations) or the memory address (for stores) where the instruction result should be written. The value field is used to hold the value of the instruction result until the instruction commits. We will see an example of ROB entries shortly. Finally, the ready field indicates that the instruction has completed execution, and the value is ready.

What happens with Store Buffers?

The ROB subsumes the store buffers. Stores still execute in two steps, but the second step is performed by instruction commit. Although the renaming function of the reservation stations is replaced by the ROB, we still need a place to buffer operations (and operands) between the time they issue and the time they begin execution.

This function is still provided by the reservation stations. Because every instruction has a position in the ROB until it commits, we tag a result using the ROB entry number rather than using the reservation station number. This tagging requires that the ROB assigned for an instruction must be tracked in the reservation station. Later in this section, we will explore an alternative implementation that uses extra registers for renaming and a queue that replaces the ROB to decide when instructions can commit.

Exceptions in Speculative Execution

- **Register Renaming**

In the presence of a ROB, the renaming function of the reservation stations is replaced by the ROB. A result is tagged using the ROB entry number rather than using the reservation station number.

- **Exceptions**

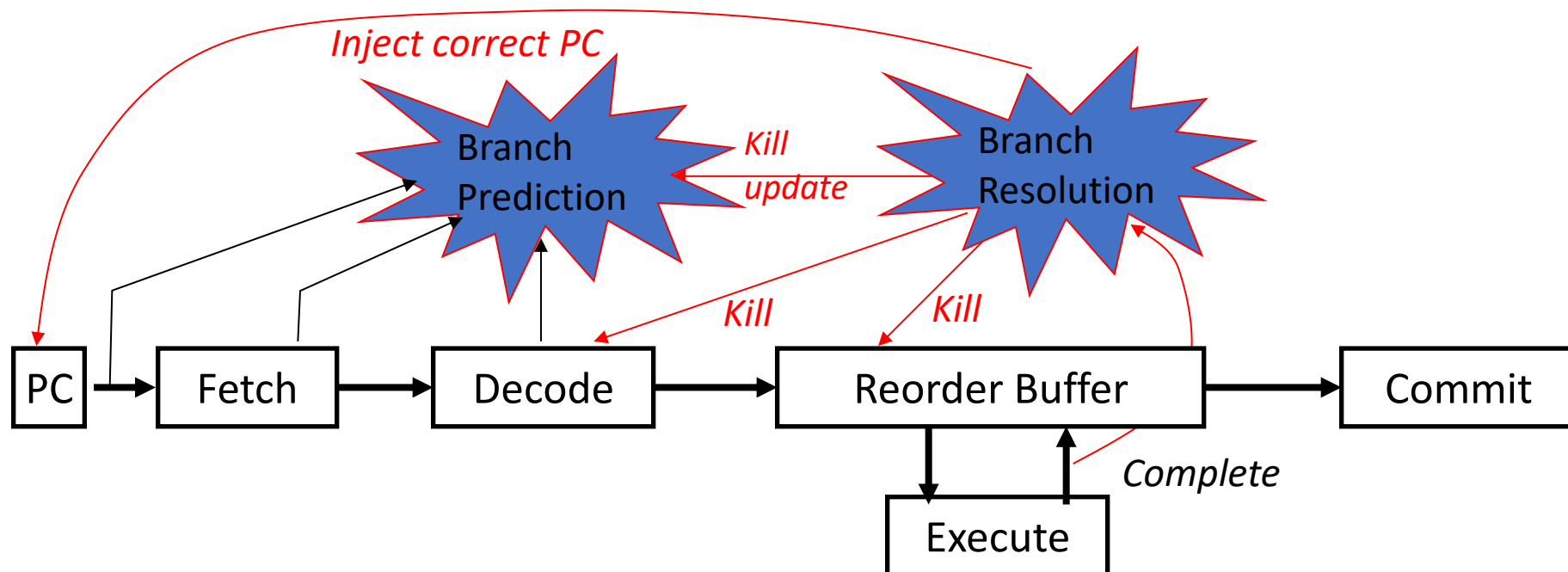
The use of a ROB with in-order instruction commit provides precise exceptions, in addition to supporting speculative execution.

- **Exceptions are handled**

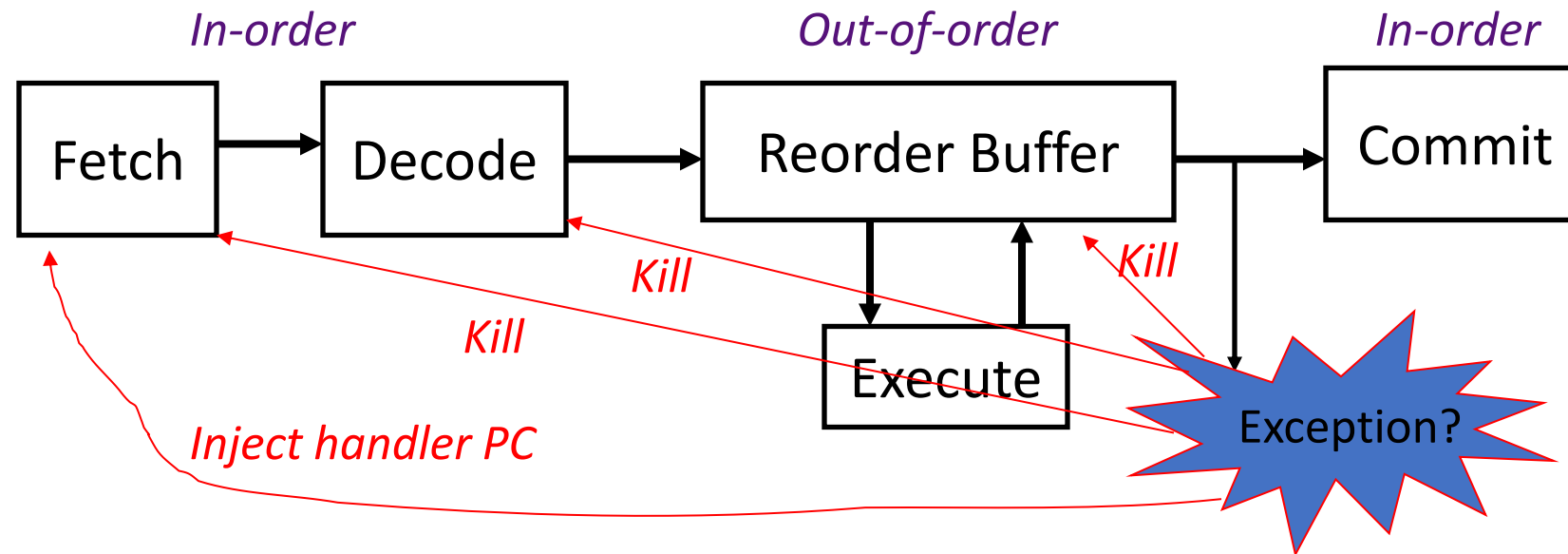
By not recognizing the exception until it is ready to commit. If a speculated instruction raises an exception, the exception is recorded in the ROB. If a branch misprediction arises and the instruction should not have been executed, the exception is flushed along with the instruction when the ROB is cleared. If the instruction reaches the head of the ROB, then we know it is no longer speculative, and the exception should really be taken.

Handling Incorrect Speculation

- Instructions following mispredicted branch, i.e. those in decode/issue buffers & RSs, are invalidated
- ROB entries of these instructions are deallocated
- Restart fetch at correct branch successor



Handling Precise Exception / Interrupt



- Instructions fetched and decoded into ROB in-order
- Execution is out-of-order → OOO completion
- Commit (write-back to architectural state, i.e., register file and memory) is in-order

Steps 1 – 2 in Dynamic Scheduler with Speculation



Apart from the three steps in the dynamic scheduler, an **additional step of commit** is introduced when there is support for speculation. **Total four steps** in a dynamic scheduler with speculation are below:

- **Issue**

- Get next instruction from FIFO queue
- If available RS entry and ROB entry, issue the instruction
- If a RS or ROB entry is not available, it becomes a structural hazard and the instruction stalls
- If an earlier instruction is not issued, then subsequent instructions cannot be issued
- If operands are available read them and store them in the reservation station\
- Operands can be read from the register file (already committed instructions) or from the ROB (executed but not yet committed)
- Operands dependent on an earlier instruction are identified by their corresponding ROB tag in the Q field and such instructions will have to stall till data becomes available

- **Execute**

- The CDB broadcasts data with an ROB tag and when an operand becomes available, store it in any reservation station waiting for it
- When all operands are ready, issue the instruction for execution
- Free the reservation station entry once the instruction moves into execution
- Loads and store are maintained in program order through the effective address
- Even with pending branches, instructions are speculatively executed

Steps 3 – 4 in Dynamic Scheduler with Speculation



- **Write result**

- Write result on CDB into reservation stations and ROB

- **Commit**

- An instruction is committed when it reaches the head of the ROB and its result is present in the buffer
- If it is an ALU or Load instruction, update the register with the result and remove the instruction from the ROB
- Committing a store is similar except that memory is updated rather than a result register
- If a branch with incorrect prediction reaches the head of the ROB, it indicates that the speculation was wrong
- ROB is flushed and execution is restarted at the correct successor of the branch
- If the branch was correctly predicted, the branch is removed from ROB
- Once an instruction commits, its entry in the ROB is reclaimed
- If the ROB fills, we simply stop issuing instructions until an entry is made free.

ROB Example 1



Let us consider the sequence of instructions below.

Assume the latencies for the Add, Mul and Div instructions to be 2, 6 and 12 respectively.

Shows how the status will look like when the **MUL.D** is ready to go to commit

L.D F6,32(R2)

L.D F2,44(R3)

MUL.D F0,F2,F4

SUB.D F8,F6,F2

DIV.D F10,F0,F6

ADD.D F6,F8,F2

Figure shows how the status tables will look like when the **MUL.D** is ready to go to commit.

Observe that though the Sub and Add instructions have written the result, they have not yet committed.

This is because the Mul instruction has not yet committed.

The Destination field designates the ROB entry that is the destination for the result produced by a particular reservation station entry.

The complete schedule is given in following Table.

Note that the instructions are **issued** in-order, **executed** out-of-order, **completed** out-of-order, but **committed** in-order.

Instruction	Issues at	Executes at	Writes result at	Commits at
L.D F6,32(R2)	1	2-3	4	5
L.D F2,44(R3)	2	3-4	5	6
MUL.D F0,F2,F4	3	6-11	12	13
SUB.D F8,F6,F2	4	6-7	8	14
DIV.D F10,F0,F6	5	13-24	25	26
ADD.D F6,F8,F2	6	9-10	11	27

‘In Order’ Commit,
Same as Instructions



Solution - contd

add is 2 clock cycles, multiply is 6 clock cycles, and divide is 12 clock cycles.

Reorder buffer						
Entry	Busy	Instruction		State	Destination	Value
1	No	L.D	F6, 32(R2)	Commit	F6	Mem[32 + Regs[R2]]
2	No	L.D	F2, 44(R3)	Commit	F2	Mem[44 + Regs[R3]]
3	Yes	MUL.D	F0, F2, F4	Write result	F0	#2 × Regs[F4]
4	Yes	SUB.D	F8, F2, F6	Write result	F8	#2 − #1
5	Yes	DIV.D	F10, F0, F6	Execute	F10	
6	Yes	ADD.D	F6, F8, F2	Write result	F6	#4 + #2

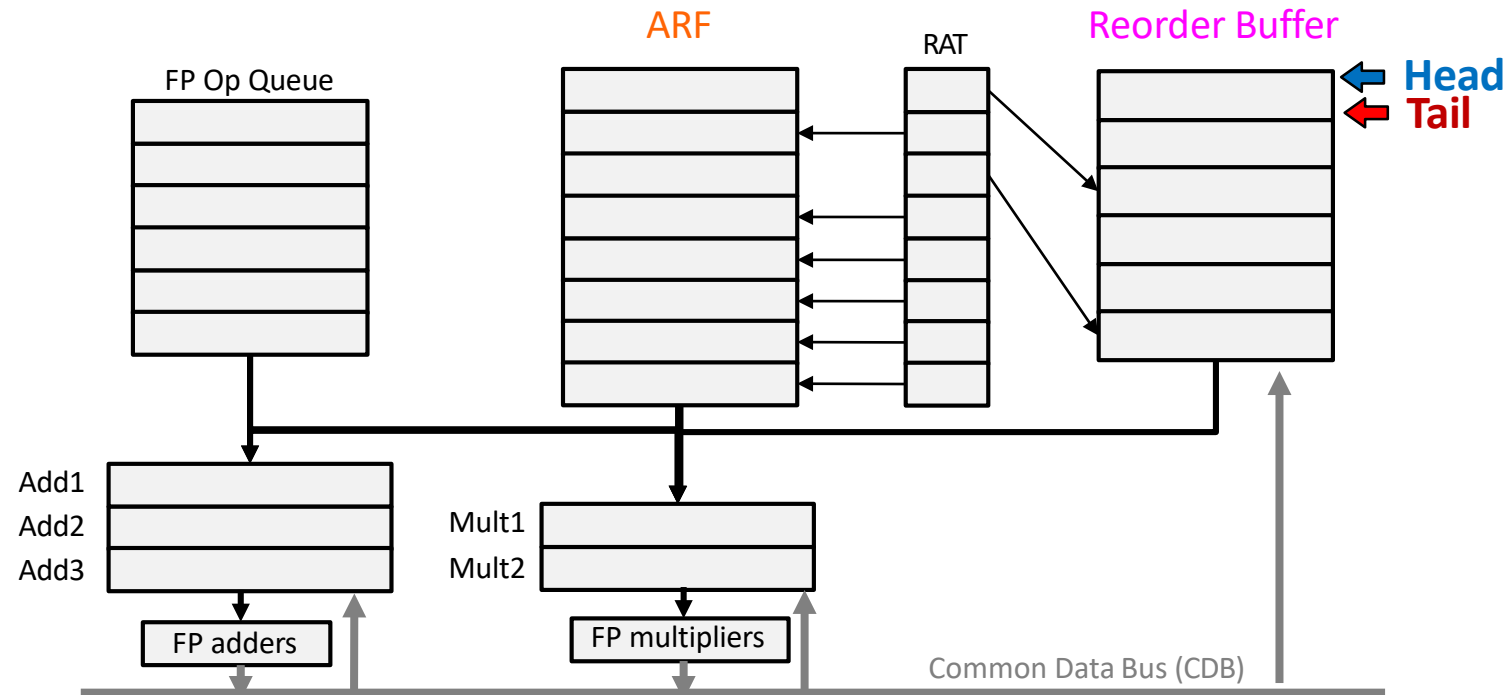
Reservation stations								
Name	Busy	Op	Vj	Vk	Qj	Qk	Dest	A
Load1	No							
Load2	No							
Add1	No							
Add2	No							
Add3	No							
Mult1	No	MUL.D	Mem[44 + Regs[R3]]	Regs[F4]			#3	
Mult2	Yes	DIV.D		Mem[32 + Regs[R2]]	#3		#5	

FP register status										
Field	F0	F1	F2	F3	F4	F5	F6	F7	F8	F10
Reorder #	3						6		4	5
Busy	Yes	No	No	No	No	No	Yes	...	Yes	Yes

Figure 3.12 At the time the MUL.D is ready to commit, only the two L.D instructions have committed, although several others have completed execution. The MUL.D is at the head of the ROB, and the two L.D instructions are there only to ease understanding. The SUB.D and ADD.D instructions will not commit until the MUL.D instruction commits, although the results of the instructions are available and can be used as sources for other instructions. The DIV.D is in execution, but has not completed solely due to its longer latency than MUL.D. The Value column indicates the value being held; the format #X is used to refer to a value field of ROB entry X. Reorder buffers 1 and 2 are actually completed but are shown for informational purposes. We do not show the entries for the load/store queue, but these entries are kept in order.

Tomasulo w/ ROB Organization

- Re-Order buffer is based on Tomasulo
- Just renamed FP register to **ARF** (Architected Register File)
- Add **Re-Order Buffer** for out-of-order results
 - Buffer is managed with two pointers (head & tail)
- RAT (Register Alias Table) keeps the register renaming info.



Four Stages of Tomasulo w/ ROB

Stage1: Issue

- Read an instruction from the buffer
- Check if resources available (RS entry & ROB entry)

- Read RAT

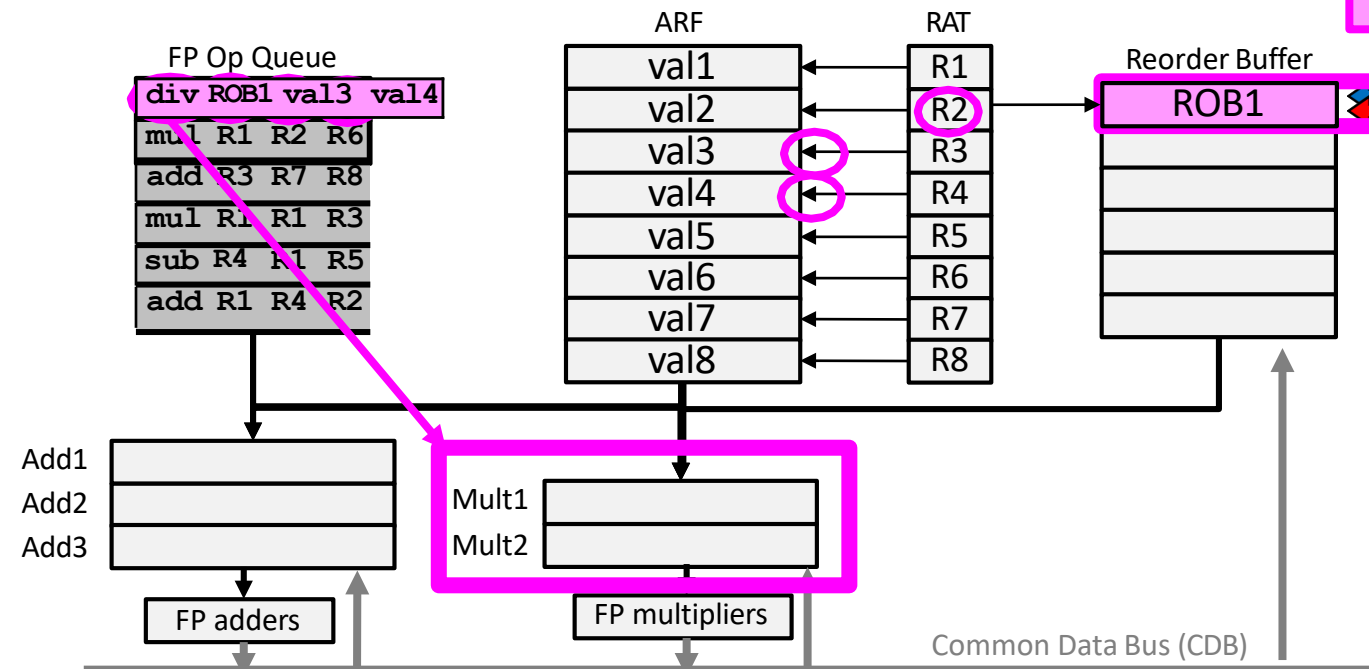
Sources: read (available) sources

Destination: update RAT

- Write to RS and ROB (the tail of ROB) & update the tail of ROB

Stall if any target resource is unavailable

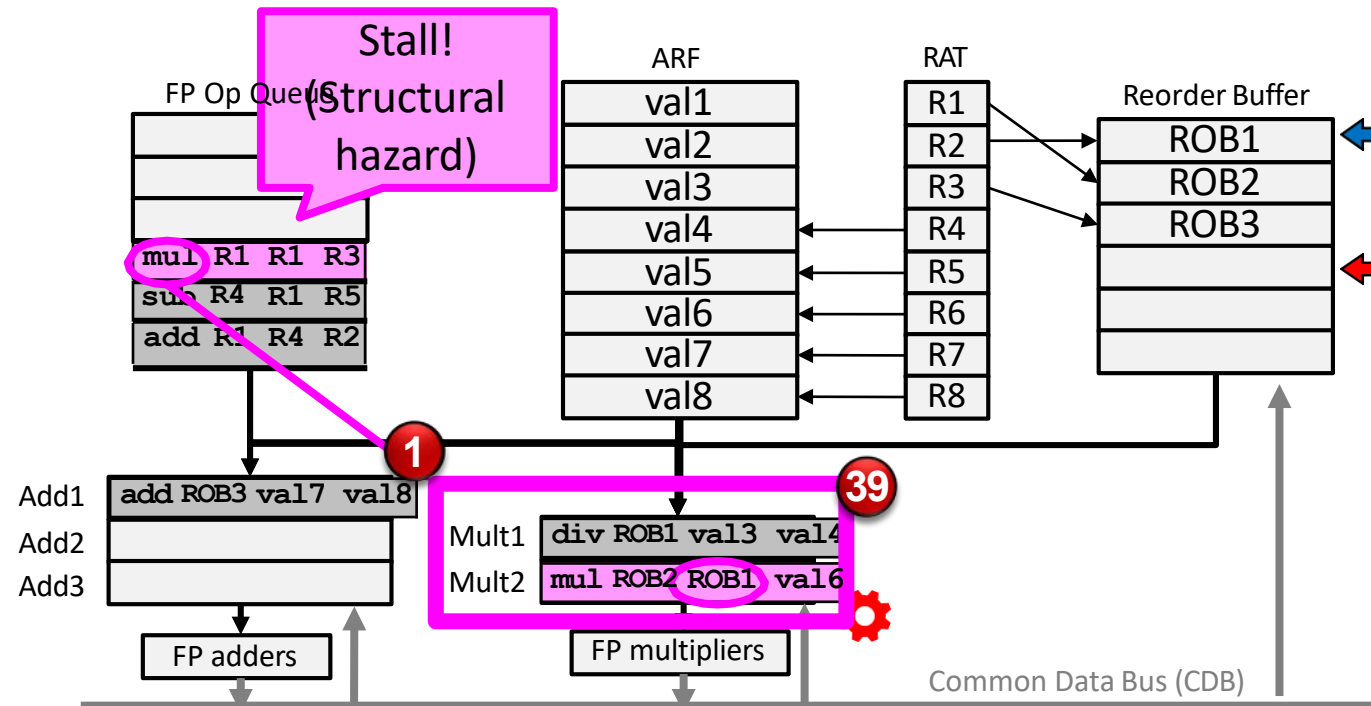
Where the tail pointer indicates



Four Stages of Tomasulo w/ ROB

Stage2: Exec

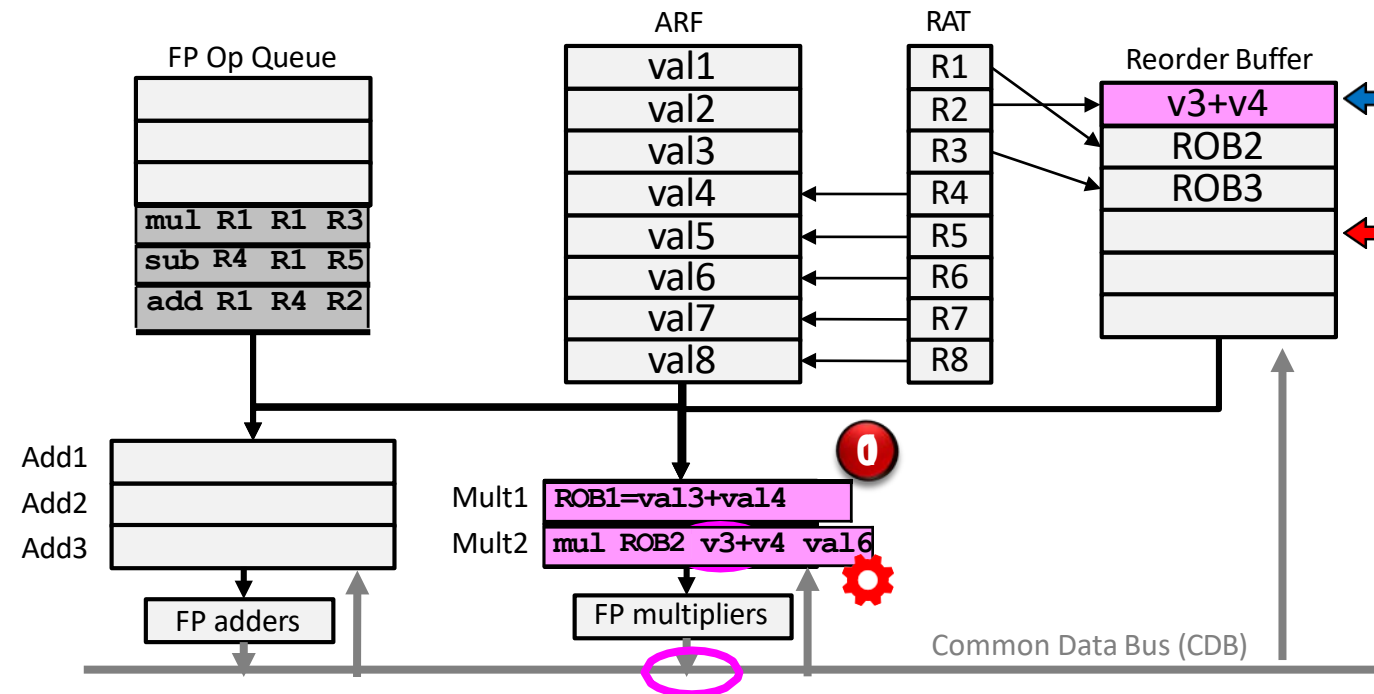
- ➔ Different behaviors based on the target instruction status
- For “a not-ready instruction” ➔ Wait for all operands to arrive
 - For “a new instruction” ➔ Compete to use the functional unit
 - For “an all-ready instruction” ➔ Execute!



Four Stages of Tomasulo w/ ROB

Stage3: Write Result

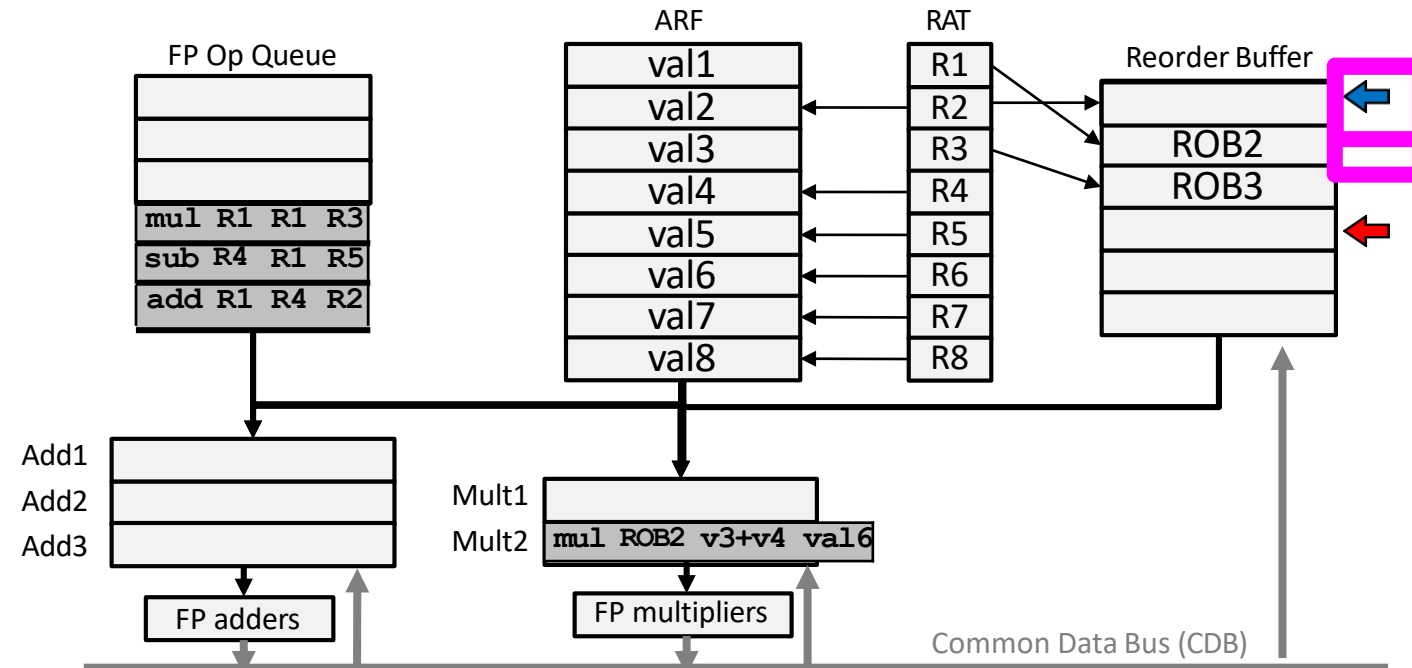
- Broadcast result on CDB
 - (any dependents will grab the value)
- Write result back to your ROB entry
 - The ARF holds the “official” register state, which we will only update in the program order
 - Mark ready/finished bit in ROB (note that this instruction has completed execution)
- Reservation station can be freed



Four Stages of Tomasulo w/ ROB

Stage4: Commit

- When the oldest instruction is completed in the ROB
 - i.e., ROB's head points to it
- Write the result (if ready/finished bit is set)
- Advance ROB's head to the next instruction
- This is what the outside world sees
 - And it's all in-order



Exploiting ILP Using Multiple Issue, VLIW, and Static Scheduling

Goal of Multiple-Issue Processors



The goal of the *multiple-issue processors*, discussed in the next few sections, is to allow multiple instructions to issue in a clock cycle. Multiple-issue processors come in three major flavors:

1. Statically scheduled superscalar processors
2. VLIW (very long instruction word) processors
3. Dynamically scheduled superscalar processors

Superpipelining vs Multiple Issue



Superpipelining

Increase the depth of the pipeline to increase the clock rate

The more stages in the pipeline, the more forwarding/hazard hardware needed and the more pipeline latch overhead (i.e., the pipeline latch accounts for a larger and larger percentage of the clock cycle time)

Multiple-Issue

Fetch (and execute) more than one instructions at one time (expand every pipeline stage to accommodate multiple instructions)

- **“Flynn bottleneck”**
 - single issue performance limit is $CPI = IPC = 1$
 - hazards + overhead $\Rightarrow CPI \geq 1$ ($IPC \leq 1$)
 - diminishing returns from superpipelining [Hrishikesh paper!]
- **Solution: issue multiple instructions per cycle**

	1	2	3	4	5	6	7
inst0	F	D	X	M	W		
inst1	F	D	X	M	W		
inst2		F	D	X	M	W	
inst3		F	D	X	M	W	

- **1st superscalar: IBM America $\rightarrow$ RS/6000 $\rightarrow$ POWER1**

- | | | | | | | | | | |
|----|----|----|-----|-----|-----|-----|-----|----|--|
| IF | ID | EX | MEM | WB | | | | | |
| IF | ID | EX | MEM | WB | | | | | |
| | IF | ID | EX | MEM | WB | | | | |
| | IF | ID | EX | MEM | WB | | | | |
| | | IF | ID | EX | MEM | WB | | | |
| | | IF | ID | EX | MEM | WB | | | |
| | | | IF | ID | EX | MEM | WB | | |
| | | | IF | ID | EX | MEM | WB | | |
| | | | | IF | ID | EX | MEM | WB | |
| | | | | IF | ID | EX | MEM | WB | |

Superscalar Processor Types



The two types of superscalar processors issue varying numbers of instructions per clock and use in-order execution if they are statically scheduled or out-of-order execution if they are dynamically scheduled.

VLIW processors, in contrast, issue a fixed number of instructions formatted either as one large instruction or as a fixed instruction packet with the parallelism among instructions explicitly indicated by the instruction. VLIW processors are inherently statically scheduled by the compiler. When Intel and HP created the IA-64 architecture, described in Appendix H, they also introduced the name EPIC (explicitly parallel instruction computer) for this architectural style.

Although statically scheduled superscalars issue a varying rather than a fixed number of instructions per clock, they are actually closer in concept to VLIWs because both approaches rely on the compiler to schedule code for the processor. Because of the diminishing advantages of a statically scheduled superscalar as the issue width grows, statically scheduled superscalars are used primarily for narrow issue widths, normally just two instructions. Beyond that width, most designers choose to implement either a VLIW or a dynamically scheduled superscalar. Because of the similarities in hardware and required compiler technology, we focus on VLIWs in this section, and we will see them again in [Chapter 7](#). The insights of this section are easily extrapolated to a statically scheduled superscalar.

5 Primary Approaches for Multi-issue Processor

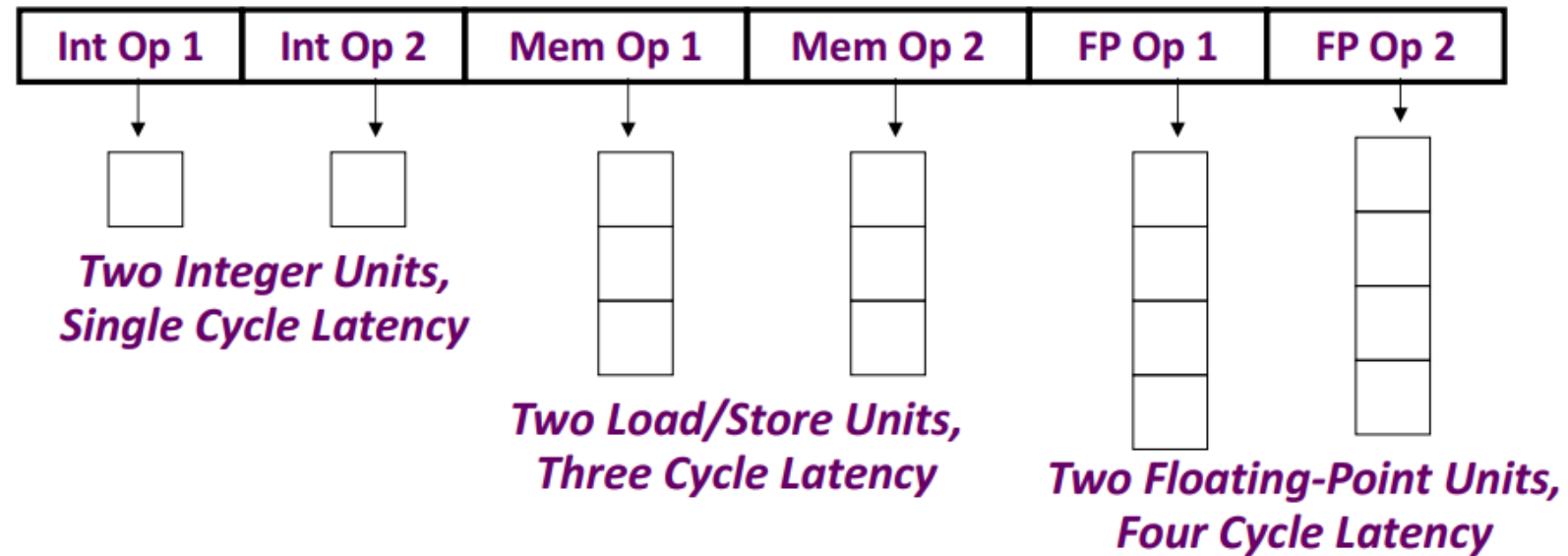
Common name	Issue structure	Hazard detection	Scheduling	Distinguishing characteristic	Examples
Superscalar (static)	Dynamic	Hardware	Static	In-order execution	Mostly in the embedded space: MIPS and ARM, including the Cortex-A53
Superscalar (dynamic)	Dynamic	Hardware	Dynamic	Some out-of-order execution, but no speculation	None at the present
Superscalar (speculative)	Dynamic	Hardware	Dynamic with speculation	Out-of-order execution with speculation	Intel Core i3, i5, i7; AMD Phenom; IBM Power 7
VLIW/LIW	Static	Primarily software	Static	All hazards determined and indicated by compiler (often implicitly)	Most examples are in signal processing, such as the TI C6x
EPIC	Primarily static	Primarily software	Mostly static	All hazards determined and indicated explicitly by the compiler	Itanium

Figure 3.19 The five primary approaches in use for multiple-issue processors and the primary characteristics that distinguish them. This chapter has focused on the hardware-intensive techniques, which are all some form of superscalar. Appendix H focuses on compiler-based approaches. The EPIC approach, as embodied in the IA-64 architecture, extends many of the concepts of the early VLIW approaches, providing a blend of static and dynamic approaches.

The Basic VLIW Approach

VLIWs use multiple, independent functional units. Rather than attempting to issue multiple, independent instructions to the units, a VLIW packages the multiple operations into one very long instruction or requires that the instructions in the issue packet satisfy the same constraints. Because there is no fundamental difference in the two approaches, we will just assume that multiple operations are placed in one instruction, as in the original VLIW approach.

Because the advantage of a VLIW increases as the maximum issue rate grows, we focus on a wider issue processor. Indeed, for simple two-issue processors, the overhead of a superscalar is probably minimal. Many designers would probably argue that a four-issue processor has manageable overhead, but as we will see later in this chapter, the growth in overhead is a major factor limiting wider issue processors.



- Multiple operations packed into one instruction
- Each operation slot is for a fixed function
- Constant operation latencies are specified
- Architecture requires guarantee of:
 - Parallelism within an instruction => no cross-operation RAW check
 - No data use before data ready => no data interlocks

22

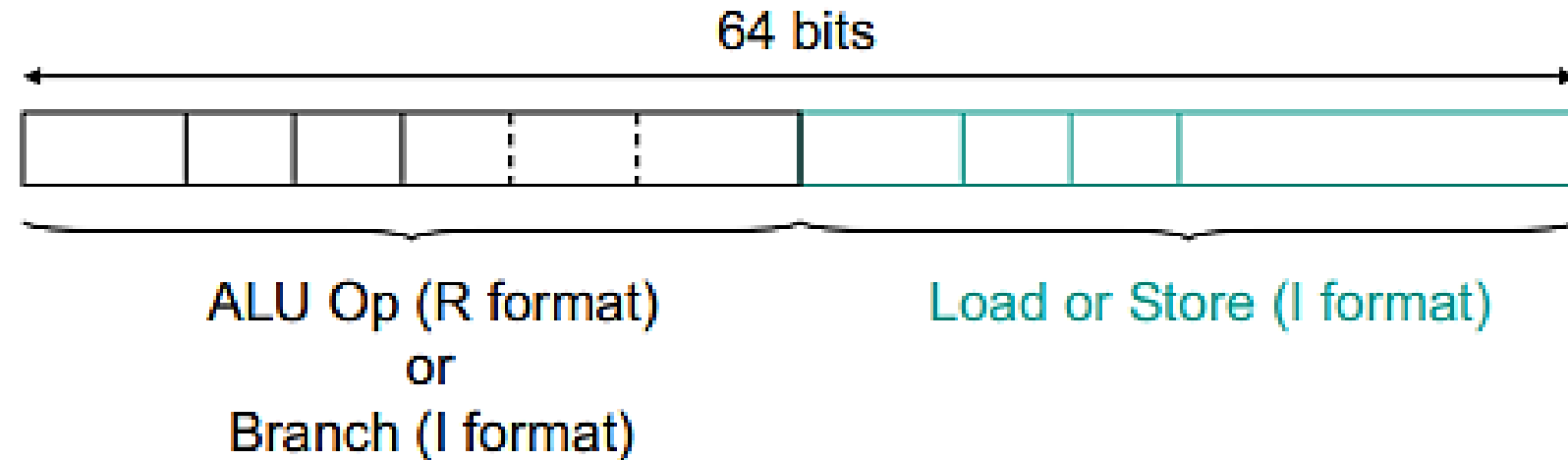
Multiple Instructions Issued

The ALU and data transfer instructions are issued at the same time.

Instruction type	Pipe stages							
ALU or branch instruction	IF	ID	EX	MEM	WB			
Load or store instruction	IF	ID	EX	MEM	WB			
ALU or branch instruction		IF	ID	EX	MEM	WB		
Load or store instruction		IF	ID	EX	MEM	WB		
ALU or branch instruction			IF	ID	EX	MEM	WB	
Load or store instruction			IF	ID	EX	MEM	WB	
ALU or branch instruction				IF	ID	EX	MEM	WB
Load or store instruction				IF	ID	EX	MEM	WB

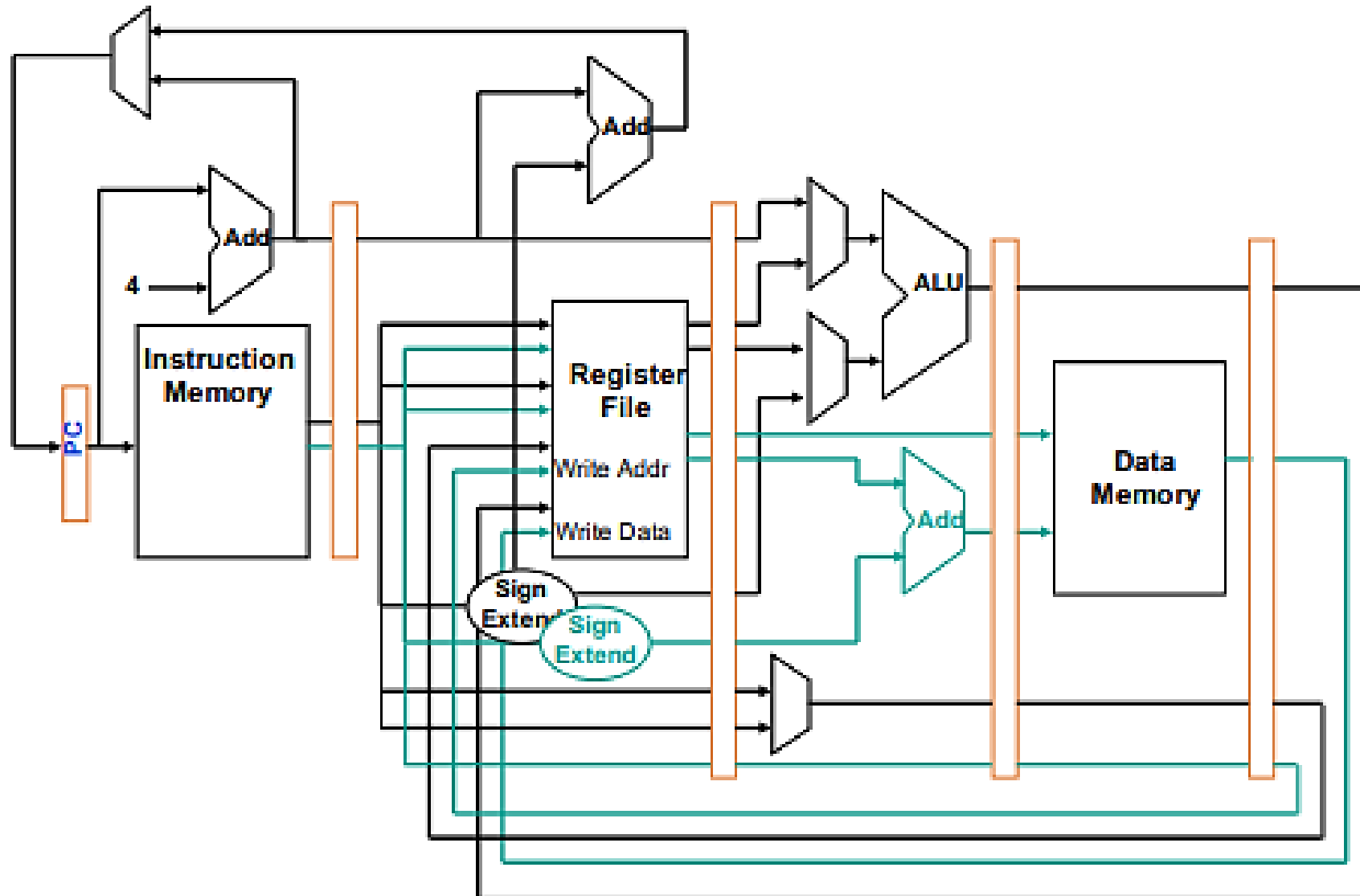
- Each “instruction” has explicit coding for multiple operations
 - In IA-64, grouping called a “packet”
 - In Transmeta, grouping called a “molecule” (with “atoms” as ops)
- Tradeoff instruction space for simple decoding
 - The long instruction word has room for many operations
 - By definition, all the operations the compiler puts in the long instruction word are independent => execute in parallel
 - E.g., 1 integer operation/branch, 2 FP ops, 2 Memory refs
 - » 16 to 24 bits per field => 5*16 or 80 bits to 5*24 or 120 bits wide
 - Need compiling technique that schedules across several branches

Consider a 2-issue RISC-V with a 2 instr bundle



- Instructions are always fetched, decoded, and issued in pairs
- If one instr of the pair can not be used, it is replaced with a noop
- Need 4 read ports and 2 write ports and a separate memory address adder

2 Issue Datapath



Parallelism needed for VLIW



Let's consider a VLIW processor with instructions that contain five operations, including one integer operation (which could also be a branch), two floating-point operations, and two memory references. The instruction would have a set of fields for each functional unit—perhaps 16–24 bits per unit, yielding an instruction length of between 80 and 120 bits. By comparison, the Intel Itanium 1 and 2 contain six operations per instruction packet (i.e., they allow concurrent issue of two three-instruction bundles, as Appendix H describes).

To keep the functional units busy, there must be enough parallelism in a code sequence to fill the available operation slots. This parallelism is uncovered by unrolling loops and scheduling the code within the single larger loop body. If the unrolling generates straight-line code, then local scheduling techniques, which operate on a single basic block, can be used. If finding and exploiting the parallelism require scheduling code across branches, a substantially more complex global scheduling algorithm must be used. Global scheduling algorithms are not only more complex in structure, but they also must deal with significantly more complicated trade-offs in optimization, because moving code across branches is expensive.

Very Important Terms

- **Dynamic Scheduling** → Out-of-order Execution
- **Speculation** → In-order Commit
- **Superscalar** → Multiple Issue

Techniques	Goals	Implementation	Addressing	Approaches
Dynamic Scheduling	Out-of-order execution	Reservation Stations, Load/Store Buffer and CDB	Data hazards (RAW, WAW, WAR)	Register renaming
Speculation	In-order commit	Branch Prediction + Reorder Buffer	Control hazards	Prediction and misprediction recovery
Superscalar /VLIW	Multiple issue	Software and Hardware	To Increase CPI	By compiler or hardware